

Waste Classification Project

Amiel H. Nava

Julia A. Reinhart

Carlos G. Cazares

CSCI 611 Applied Machine Learning Spring 2026

Professor Dr. Bo Shen

May 15, 2026

Overall Context

Objective

This project aims to gain insights into the process of further training pre-trained CNN architecture models to predict a small, specific custom dataset and then applying that model in a real-world scenario. This scenario arises when we need an application for a classification model, but lack the necessary resources such as time and computational power to design, build, train, and gather a substantial amount of data to train a classification model from scratch. This project will compare and contrast multiple CNN architectures to analyze their training times and prediction accuracy. Along with comparing models, this project will display several Machine Learning techniques used to increase model prediction accuracy and efficiency.

Brief Summary

We utilized the RealWaste dataset to train five models capable of classifying trash into nine distinct categories. Prior to training, we established a data preparation pipeline to enable the models to accept images as input. After implementing the data preparation pipeline for both non-YOLO and YOLO models, we commenced training the pre-trained models by freezing the weights of all layers except the last one and adjusting the output size to nine, corresponding to the number of layers in the RealWaste dataset. To enhance the accuracy of our models, we employed various techniques, including data augmentation, adjusting the learning rate based on the depth of the layers from the top, experimenting with different optimizers and learning schedulers, and gradually unfreezing the weights in deeper layers until no significant improvements were observed. In our project, we employed several techniques to minimize the training time for our models. Once we obtained the optimal weights for each model, we saved them and later loaded them to compare and contrast their performance. Finally, we utilized the best saved models and integrated them into an applied mobile application for real-time trash label detection.

Overall Context and Relevant Work

This project centers on image classification of waste materials using the RealWaste Dataset. The main objective is to classify waste into 9 categories including Cardboard, Food Organics, Glass, Metal, Miscellaneous Trash, Paper, Plastic, Textile Trash, and Vegetation. The dataset contains a total of 4,752

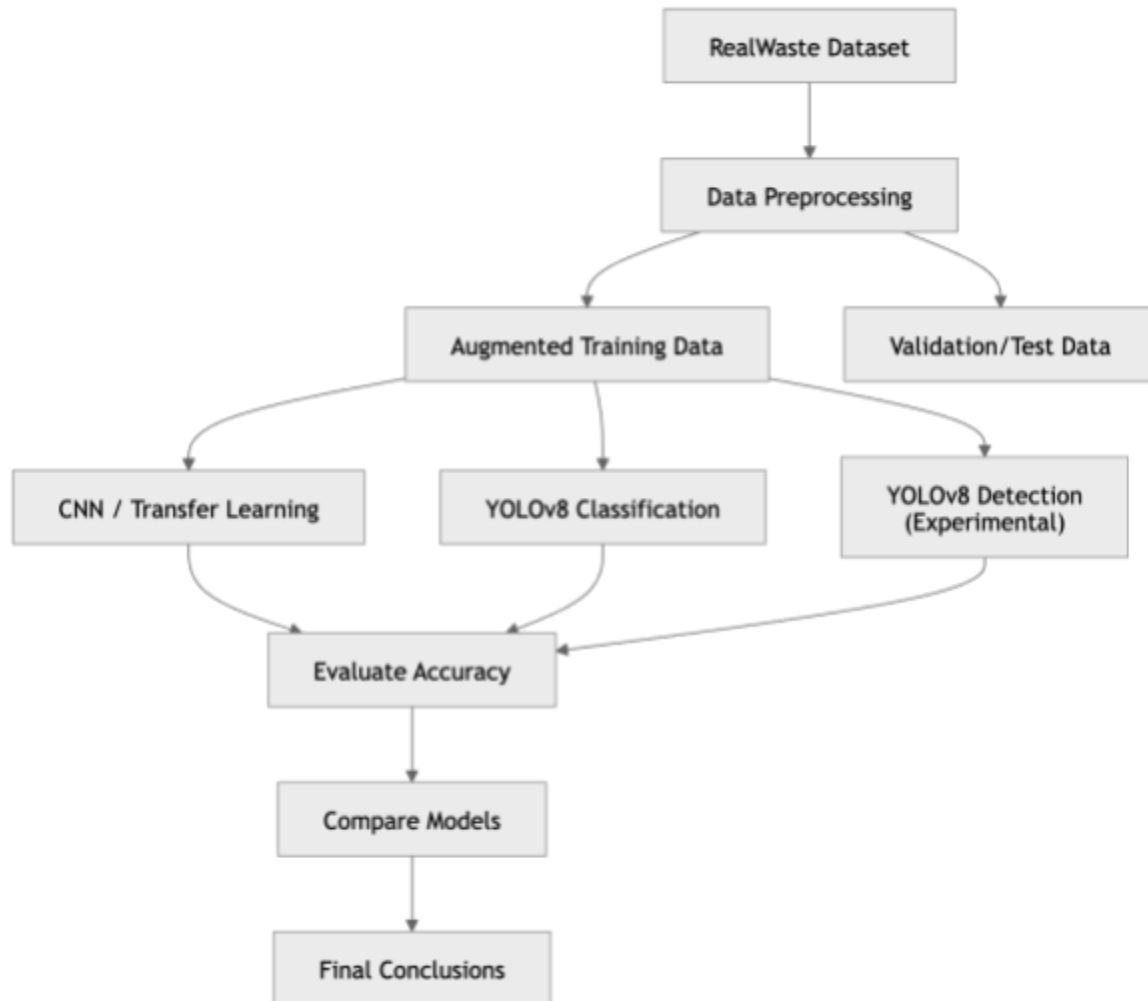
labeled images, which is on the smaller side for deep learning. Due to limited data, data augmentation and transfer data were implemented to increase the accuracy of the model. This project encompasses Image Classification using CNNs and YOLOv8 classification in addition to Object Detection using YOLOv8 detection with bounding boxes. Different models that were pretrained were used including MobileNet, EfficientNet, YOLOv8 classification, and YOLOv8 detection.

Continued on the next page one sample image from each category is shown.



High-Level Framework

Below is a diagram of the architecture of our full solution:



This project follows a multi-model comparative approach including five models - three Baseline CNN models using PyTorch, YOLOv8 for classification, and YOLOv8 for detection. Data is split into training, validation, and test sets. Models are evaluated based on accuracy, loss curves, and confusion matrix. Final results are compared to determine the best-performing model. See the test results and analysis section for more detail on each of the three models.

Uniqueness of our Solution

Unlike standard classification projects, this project combines classification and detection approaches from a small dataset compared to other classification projects with larger datasets. Additionally to classification, we introduce automatic bounding box generation using OpenCV. This adds originality in that we had to use a creative approach to maximize the outcome with our limited data. We implement YOLOv8 in two different modes with classification for a higher potential for high accuracy and detection to expand upon our existing data. We also used three classical classification models to compete with each other in order to compete with

the final YOLO model. Finally, our solution utilizes a CoreML export for mobile deployment and we have integrated our best model into a real-world demo using a mobile application on the iPhone.

Detailed Aspects

Research

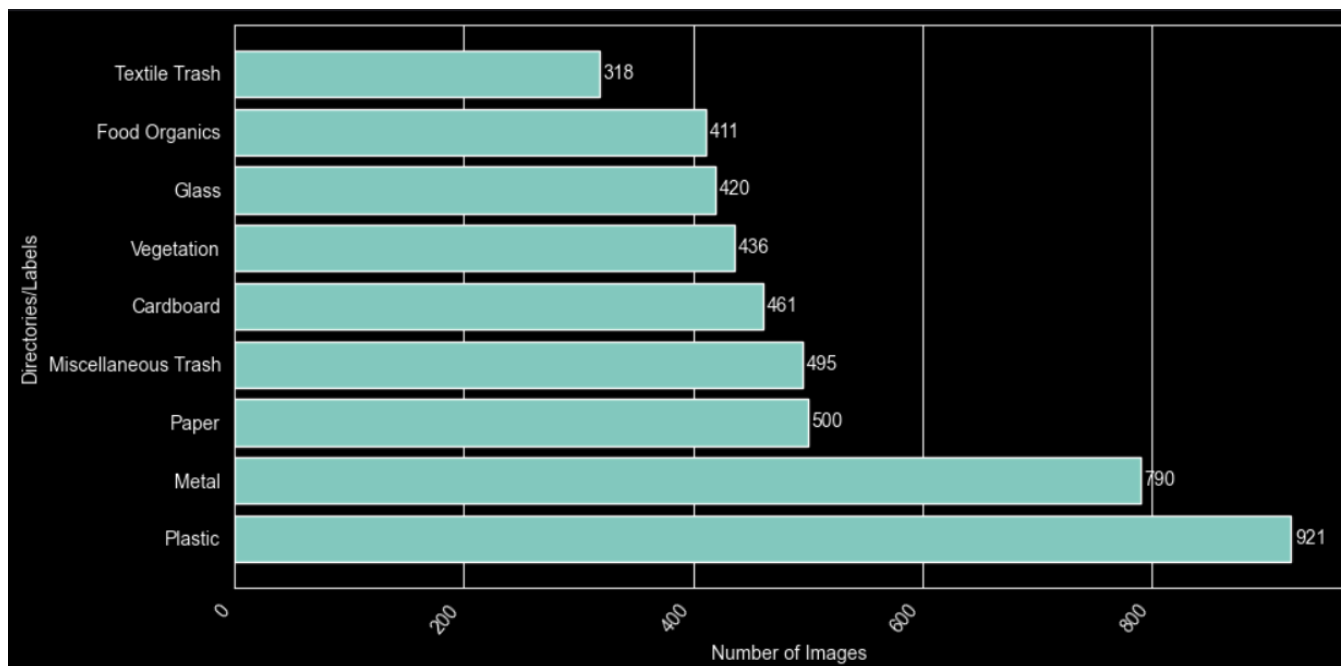
We plan our project with the guidance of “Hands-On Machine Learning with Scikit-Learn and PyTorch” Chapter 2 End-to-End Machine Learning Project, but with several modifications to adjust our specifications and goals [1]. For utilizing Machine Learning libraries such as PyTorch, Torchvision, Scikit-Learn, and others, we reference their documentation. Another reference we used for using PyTorch with CNN models was “Hands-On Machine Learning with Scikit-Learn and PyTorch” Chapter 11-12. The data preparation and fine-tuning resources we used were in the book “Machine Learning with PyTorch and Scikit”[2]. Both books provide great guidance on using and understanding Machine Learning libraries and their best practices.

For using YOLO object detection, we reference Ultralytics documentation[4]. We first tinkered with the open-source project in GitHub for YOLOv3 to understand the implementation and do basic tests[3]. After having a general understanding of how YOLO architecture works under the hood, we moved to using Ultralytics' pretrained YOLO model.

Data Preparation

We utilize UCI's image dataset named RealWaste for the training of our multiple models[6]. This data set consists of 4,752 images of waste materials. Those materials are classified in the table below along with the amount of images. One important aspect to our dataset to notice is how imbalanced the labels/classes are which may take an effect on our results.

Material	Number of Images
Cardboard	461
Food Organics	411
Glass	420
Metal	790
Miscellaneous Trash	495
Paper	500
Plastic	921
Textile Trash	318
Vegetation	436



Images Per Label Bar Graph Sorted

The design for our Jupyter Notebook was to be able to be run on any environment that had Python installed, which includes Google Colab with little or no additional configuration. This meant avoiding manually downloading and placing the data in the environment's directory. Instead, we implemented a function to check if the dataset was already loaded into the directory named data, which is the Jupyter Notebook standard to keep any persistent data relating to any data generated in the Notebook. If the function did not find such data, then it would download the data from the UCI's archive. Alternatively, the data can be run locally on one's computer using a virtual environment and detailed instructions on GitHub README file or in the Jupyter Notebook section named Local Runtime Setup.

```
real_waste_path = load_real_waste_data()
```

Loading Real Waste dataset...
Real Waste dataset loaded.

Notebook Section: Data->Get the Data->Load/Download RealWaste Dataset

To display the data we will be using for this project, we created a section in our Jupyter Notebook with the heading name Display RealWaste. After loading the dataset or confirming an existing dataset, we display the directory to show what type of directory structure we are dealing with and display a bar graph with the metadata values from the table above. Each directory name in the dataset represents the label for the images inside that directory. We also added a section to randomly display an image for each label in the dataset.

Given the directory structure of the RealWaste dataset, we decided to use TorchVision's ImageFolder to automatically set up a DataSet from the root path of the dataset's directory. The dataset class stores the sample and its corresponding labels to be iterated by a class DataLoader[7]. That directory structure is displayed below:

RealWaste

- Paper
- Metal
- Cardboard
- Food Organics
- Glass
- Vegetation
- Textile Trash
- Miscellaneous Trash
- Plastic

Using the ImageFolder API, we split the data into a standard split, which is training, validation, and testing. We also used the library `train_test_split` from scikit-learn to stratify our split. We used stratified sampling to ensure the training, validation, and test datasets maintain the same proportion.

Test Results and Analysis

We examine three main models and compare the results of our findings here.

1. Non-YOLO CNN / Transfer Learning Notebook

First we built a Non-YOLO CNN Transfer Learning Notebook. The non-YOLO notebook solves RealWaste as a single-label image classification task. Each image is assumed to contain a total of one main waste object, and in turn, the model predicts a single class from the following total classes: Cardboard, Food Organics, Glass, Metal, Miscellaneous Trash, Paper, Plastic, Textile Trash, or Vegetation. The dataset is downloaded from the UCI RealWaste source organized in subfolders by waste type. The baseline dataset contains a directory tree of class folders, number of images per label, and a random sample of images from each class. The model uses CrossEntropyLoss, Adam optimizer, training and validation loss tracking, and device selection for CUDA, MPS, or CPU. This approach is a strong baseline because it is simpler than object detection and directly matches the RealWaste folder-label structure.

The notebook uses transfer learning, meaning the model starts from pretrained weights from the ImageNet dataset. We reuse the lower layers of these pretrained models and retrained the upper layers of the models to learn our unique dataset that may not have been seen in the ImageNet dataset. Based on training and validation losses, we further unfreeze the upper layers until we see no gradual improvements. In other words, we use the already trained lower layers that have already been trained and optimized to extract general features of an object. We then have to train the upper layers to focus on features that make our labels unique and refocus its output or final classification to match the dimension of our class labels in our dataset. The final

classification layer is replaced so the model outputs 9 class scores instead of ImageNet's 1,000 original classes.

Select Model and Download Weights

File display

```
import torchvision

mnv3_weights = torchvision.models.MobileNet_V3_Small_Weights.IMAGENET1K_V1
mnv3_model = torchvision.models.mobilenet_v3_small(weights=mnv3_weights) # Set pretrained weights
prepare_model_for_transfer_learning(mnv3_model, len(class_names))
```

Preparing model (MobileNetV3) for transfer learning.

Freezing all weights...

Unfreezing weights in classifier...

Replacing default outer layer in MobileNetV3: Linear(in_features=1024, out_features=1000, bias=True)

Outer layer in MobileNetV3 replaced to : Linear(in_features=1024, out_features=9, bias=True)

Replacing the Output Layer Sample

To coincide with ImageNet standards, images are resized to 224 x 224 converted to tensors, and normalized using the following code:

```
val_test_transform = transforms.Compose([
    transforms.Resize((224, 224)), # Recommended 224x224 for CNNs
    transforms.ToTensor(),         # Convert Numpy/Image to Tensor
    # Normalize color channels
    transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5]),
])
```

Our baseline model does not include data augmentation or a stratified split process, and our test accuracy for the baseline model is 51% as shown in the following screenshot below. Note that only one epoch was used for the baseline model as it served as a starting point for how we could improve the test accuracy from there.

```
test_model(mnv3_model, test_loader, mnv3_config, class_names, device)
```

Test Loss: 1.433503

Test Accuracy of Cardboard: 33% (18/53)

Test Accuracy of Food Organics: 61% (24/39)

Test Accuracy of Glass: 11% (5/42)

Test Accuracy of Metal: 68% (53/77)

Test Accuracy of Miscellaneous Trash: 55% (29/52)

Test Accuracy of Paper: 50% (27/53)

Test Accuracy of Plastic: 42% (36/84)

Test Accuracy of Textile Trash: 21% (6/28)

Test Accuracy of Vegetation: 93% (45/48)

Test Accuracy (Overall): 51% (243/476)

From there, the first hyperparameter we experimented with was modifying the random data split to a stratified data split. Since our training dataset was so small, and the proportion of training data within each category was not equal or even approximately equal, a stratified data split was chosen so that the proportion of training data within each waste type would remain equal. In order to perform this task, the below code for a random split shown on the next page:

```
indices = torch.randperm(
total_samples,
generator=torch.Generator().manual_seed(seed)
).tolist()

train_indices = indices[:train_size]
validation_indices = indices[train_size:train_size + validation_size]
test_indices = indices[train_size + validation_size:]
```

Was replaced with the following code using a stratified split here:

```

targets = base_dataset.targets
indices = list(range(total_samples))

train_indices, temp_indices = train_test_split(
    indices,
    train_size=train_ratio,
    stratify=targets,
    random_state=seed
)

temp_targets = [targets[i] for i in temp_indices]

validation_indices, test_indices = train_test_split(
    temp_indices,
    test_size=0.5,
    stratify=temp_targets,
    random_state=seed
)

```

In making that single change of only replacing the random split with a stratified split, we were able to increase the model accuracy from 51% to 58%. Note that the number of epochs remained fixed at 1 epoch. Attached is a screenshot illustrating this improvement beginning on the next page:

```
test_model(mnv3_model, test_loader, mnv3_config, class_names, device)
```

Test Loss: 1.183698

Test Accuracy of Cardboard: 58% (27/46)
 Test Accuracy of Food Organics: 78% (32/41)
 Test Accuracy of Glass: 21% (9/42)
 Test Accuracy of Metal: 83% (66/79)
 Test Accuracy of Miscellaneous Trash: 20% (10/50)
 Test Accuracy of Paper: 56% (28/50)
 Test Accuracy of Plastic: 74% (69/93)
 Test Accuracy of Textile Trash: 18% (6/32)
 Test Accuracy of Vegetation: 72% (31/43)

Test Accuracy (Overall): 58% (278/476)

Next, code was implemented to save the best model regardless of the number of total epochs run so that we could experiment also with the total number of epochs. This saved model is later used to compare each model in order to use that model to compete with the YOLO model. Code is shown for saving the best model here:

```
if avg_val_loss < best_val_loss:
    best_val_loss = avg_val_loss
    best_epoch = epoch + 1
    best_model_weights = copy.deepcopy(model.state_dict())

    torch.save({
        "epoch": best_epoch,
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "best_val_loss": best_val_loss,
        "config": config,
    }, checkpoint_path)

    print(f"New best model saved at epoch {best_epoch} with val loss {best_val_loss:.4f}")
```

Next, data augmentation was incorporated using the following code shown on the next page:

```
def get_augmented_transform() -> transforms.Compose:
    return transforms.Compose([
        transforms.RandomResizedCrop(224, scale=(0.75, 1.0)),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomRotation(15),
        transforms.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.10, hue=0.02),
        transforms.RandomPerspective(distortion_scale=0.15, p=0.2),
        transforms.ToTensor(),          # Convert Numpy/Image to Tensor
        # Normalize color channels
        transforms.Normalize(NORMALIZE_MEAN, NORMALIZE_STD),
    ])
```

Since knowing that our dataset is small as stated in the data preparation section in this report, we utilize data augmentation to artificially increase the size of our training dataset. The data augmentation techniques are, resizing an image, flipping an image, rotating an image, adjusting the color of an image slightly, and distorting an image to give the illusion of a different perspective. This data augmentation is used only for the training set, not the validation or test sets. This helps reduce overfitting because the dataset is relatively small. We would be using this augmentation and similar configurations for MobileNet, EfficientNet, and GoogleNet models.

Combining data augmentation, a stratified split, and a total of 30 epochs saving the best MobileNet model, we were able to achieve a test accuracy of 84%. Below is a screenshot illustrating this:


```
test_model(mnv3_model, mnv3_test, mnv3_config, class_names, device)
```

Test Loss: 0.485128

Test Accuracy of Cardboard: 84% (39/46)

Test Accuracy of Food Organics: 97% (40/41)

Test Accuracy of Glass: 85% (36/42)

Test Accuracy of Metal: 84% (67/79)

Test Accuracy of Miscellaneous Trash: 58% (29/50)

Test Accuracy of Paper: 92% (46/50)

Test Accuracy of Plastic: 83% (78/93)

Test Accuracy of Textile Trash: 81% (26/32)

Test Accuracy of Vegetation: 95% (41/43)

Test Accuracy (Overall): 84% (402/476)

Then, using the following code below we were able to unfreeze top layers and achieve a test accuracy of 86%. This is an increase of two percent, which is a modest gain. We also unfreeze the top layers similarly with EfficientB0 and GoogleNet.

To achieve this, we added the following code below to unfreeze the top layers:

```
for param in mnv3_model.features[-2:].parameters():
    param.requires_grad = True

mnv3_finetune_config = TrainConfig(
    epochs=30,
    lr=0.00003,
    loss_function=nn.CrossEntropyLoss(),
    optimizer_class=torch.optim.AdamW,
    optimizer_kwargs={"weight_decay": 0.01},
    scheduler=torch.optim.lr_scheduler.StepLR,
    scheduler_kwargs={"step_size": 4, "gamma": 0.5},
    print_every_n_epochs=1,
)
```

Unfreezing Top Layers for the MobileNetV3 Model

The results we obtained can be found below:

```
test_model(mnv3_model, mnv3_test, mnv3_finetune_config, class_names, device)
```

```
Test Loss: 0.392351
```

```
Test Accuracy of Cardboard: 86% (40/46)
```

```
Test Accuracy of Food Organics: 97% (40/41)
```

```
Test Accuracy of Glass: 80% (34/42)
```

```
Test Accuracy of Metal: 88% (70/79)
```

```
Test Accuracy of Miscellaneous Trash: 68% (34/50)
```

```
Test Accuracy of Paper: 86% (43/50)
```

```
Test Accuracy of Plastic: 87% (81/93)
```

```
Test Accuracy of Textile Trash: 81% (26/32)
```

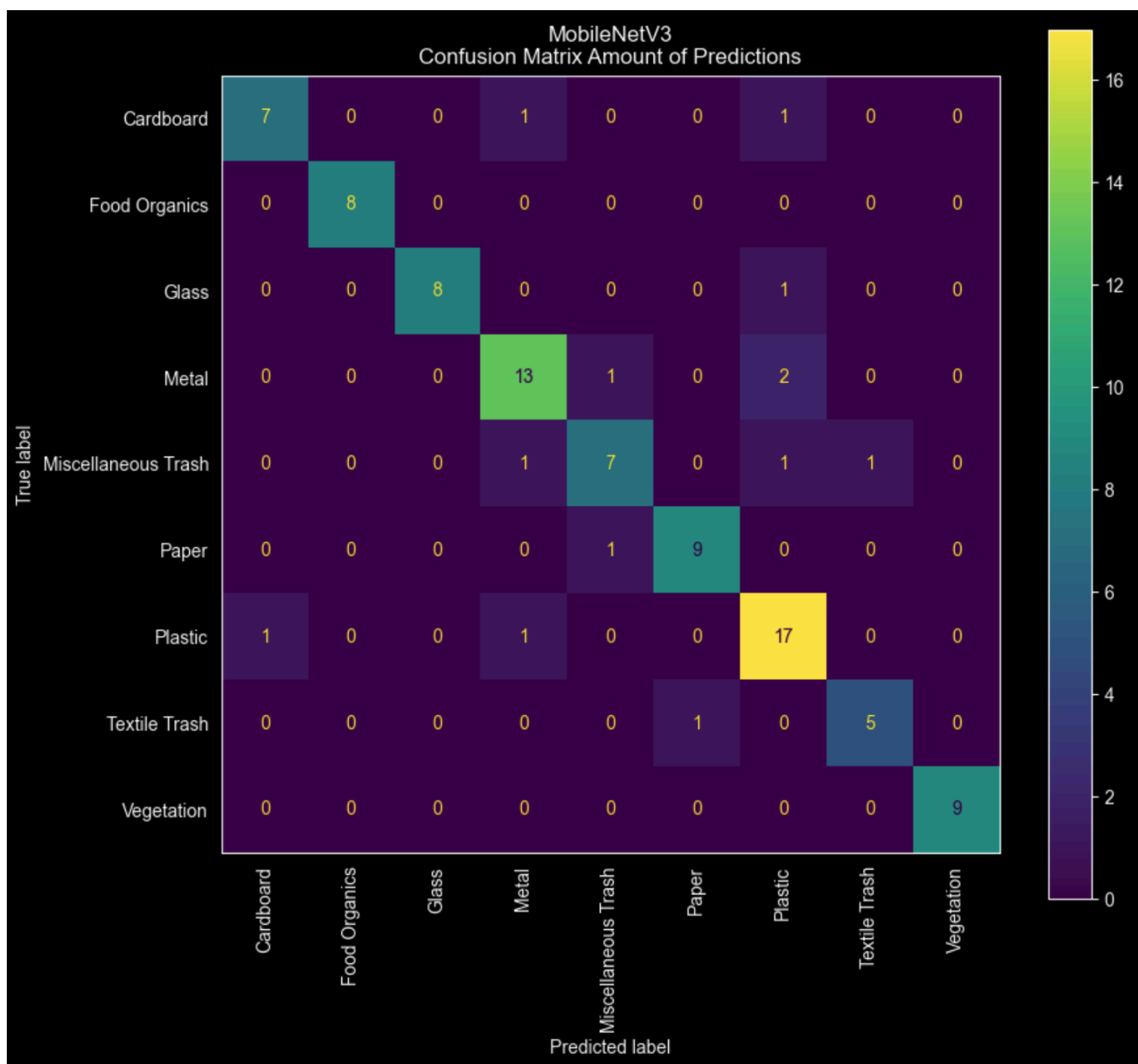
```
Test Accuracy of Vegetation: 97% (42/43)
```

```
Test Accuracy (Overall): 86% (410/476)
```

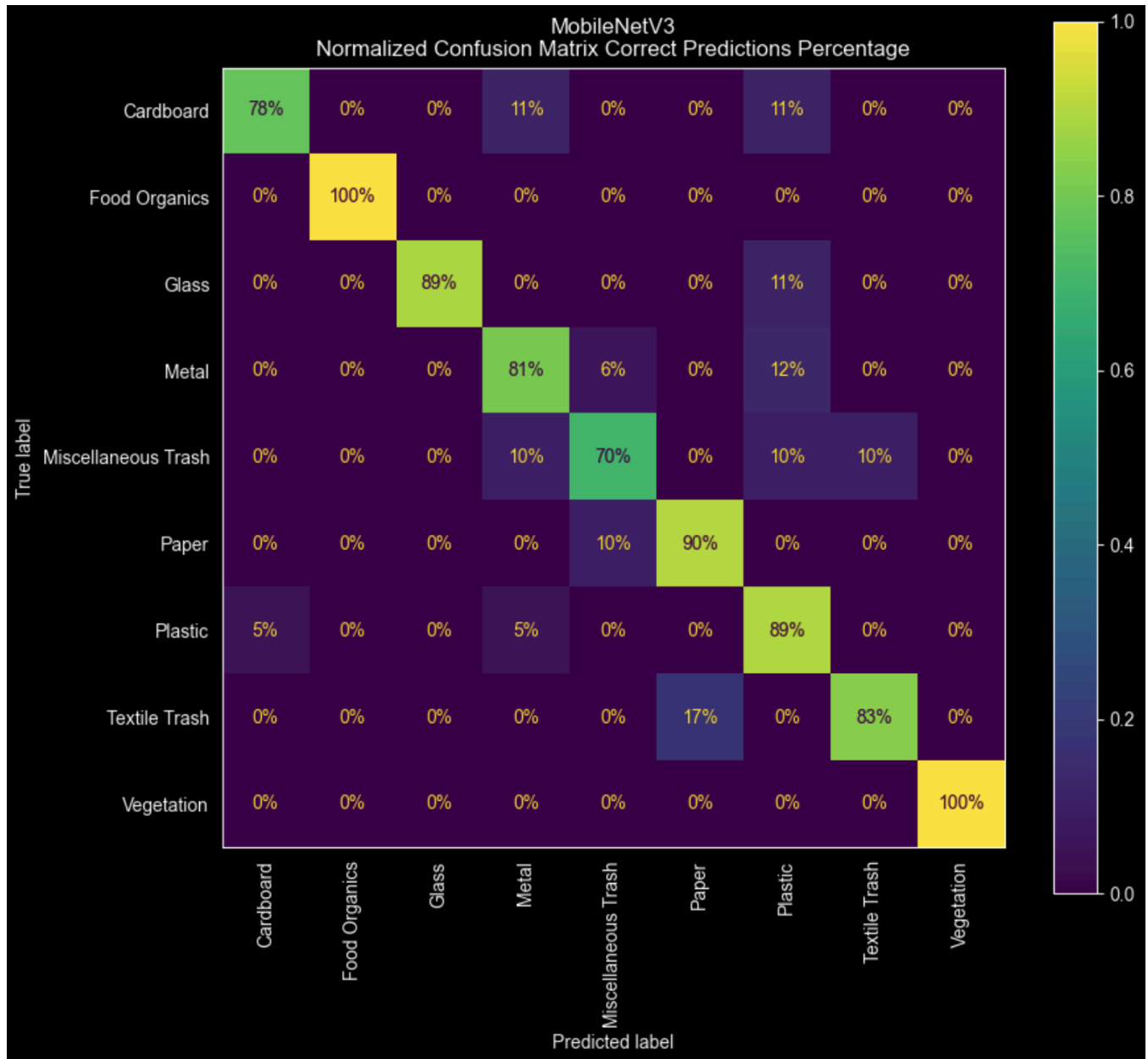
Since we achieved far greater gains by combining data augmentation and a stratified split than we did with unfreezing the top layers, this suggests that data augmentation and a stratified split had a far greater benefit than simply unfreezing the top layers. Data augmentation and stratified split were now going to be our default starting dataset for future model training.

MobileNetV3 Results

Now, we examine the confusion matrix for our MobileNet Model. A confusion matrix portrays the predicted category on the x-axis and contrasts it with the actual category on the y-axis. In each square, totals are shown that represent the total number of images that were predicted to go in the given category but were actually a member of the actual category represented in each ordered cell in the matrix. The cells in the diagonal matrix are where the model made the correct prediction, and the cells outside the diagonal matrix are where cells made an incorrect prediction or misclassified a label. Below we show the confusion matrix for our MobileNet Model:



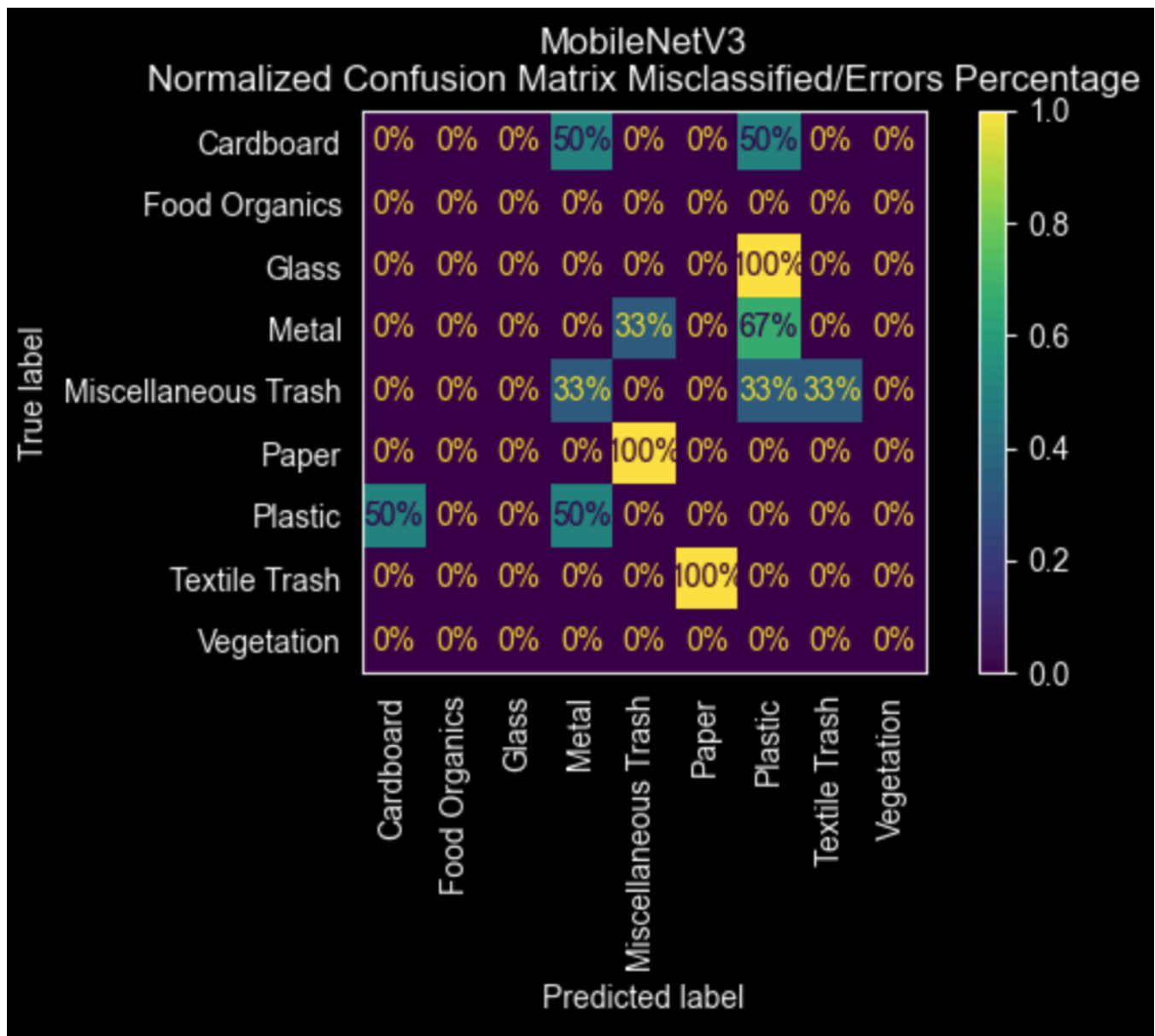
To make this matrix easier to understand, percentages can replace the values in the matrix. This is known as a normalized confusion matrix where values are normalized to be within 0 to 100 percent. From the normalized matrix, in each square, percentages are shown that represent the percentage of images that were predicted to go in the given category but were actually a member of the actual category represented in each ordered cell in the matrix. Below is our normalized matrix for the MobileNet Model:



From this matrix, we can determine that in the cell (predicted, actual) represented by the ordered pair (plastic, glass), that the model incorrectly predicts an object is plastic when it is actually glass 11% of the time. To combat this deficiency, more images need to be provided in the training data that better differentiate plastic from glass since both categories can look alike. Interestingly, from the opposite ordered pair, (glass, plastic), an incorrectly predicted value of glass when an item is actually plastic does not occur. Another area of interest is the high confidence this model has for the labels Food Organics and Vegetation, which could potentially mean that the original model trained on the ImageNet dataset had a large set of plants and food labels to be trained on. Also note that the lowest prediction was Miscellaneous Trash label which is a broad category that doesn't fall in the other eight labels in our dataset. More research can be done to further investigate this area in the model's confusion.

Overall in examining this matrix, our model performs quite well in that the highest percentages in the matrix, which average in the range of the 80th percentile, correctly predict the true category of the waste type. An inverse relationship can be shown by plotting the error in the confusion matrix which performs the opposite operation using the loss formula in our model. It detects the percentage of approximate error where objects are misclassified into an object of the wrong category.

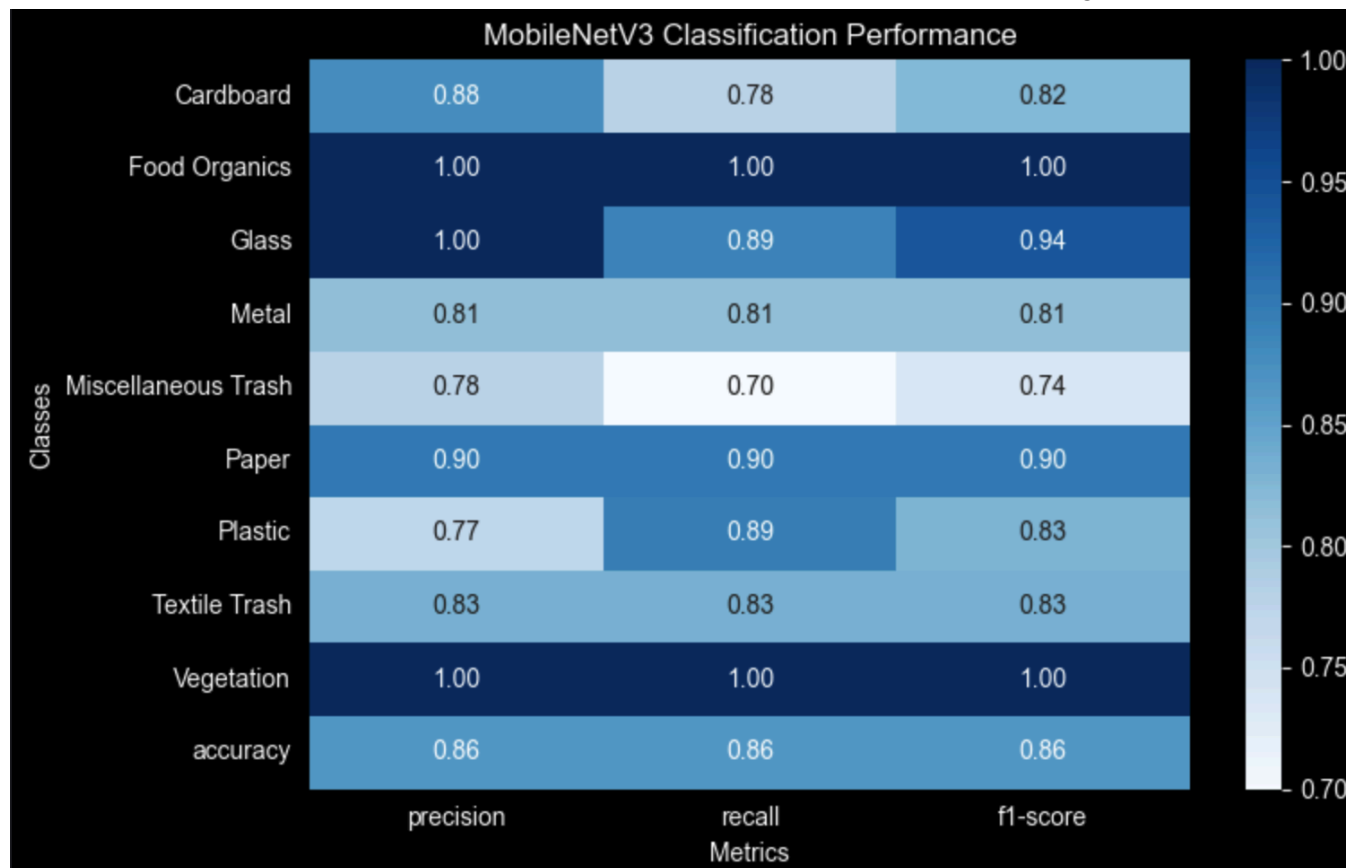
We examine this error confusion matrix below:



The error matrix shows the distribution of incorrect predictions after removing or emphasizing the correct diagonal predictions. This helps us focus only on where the model is making mistakes. It does not represent the training loss formula; instead, it visualizes misclassification patterns between classes.

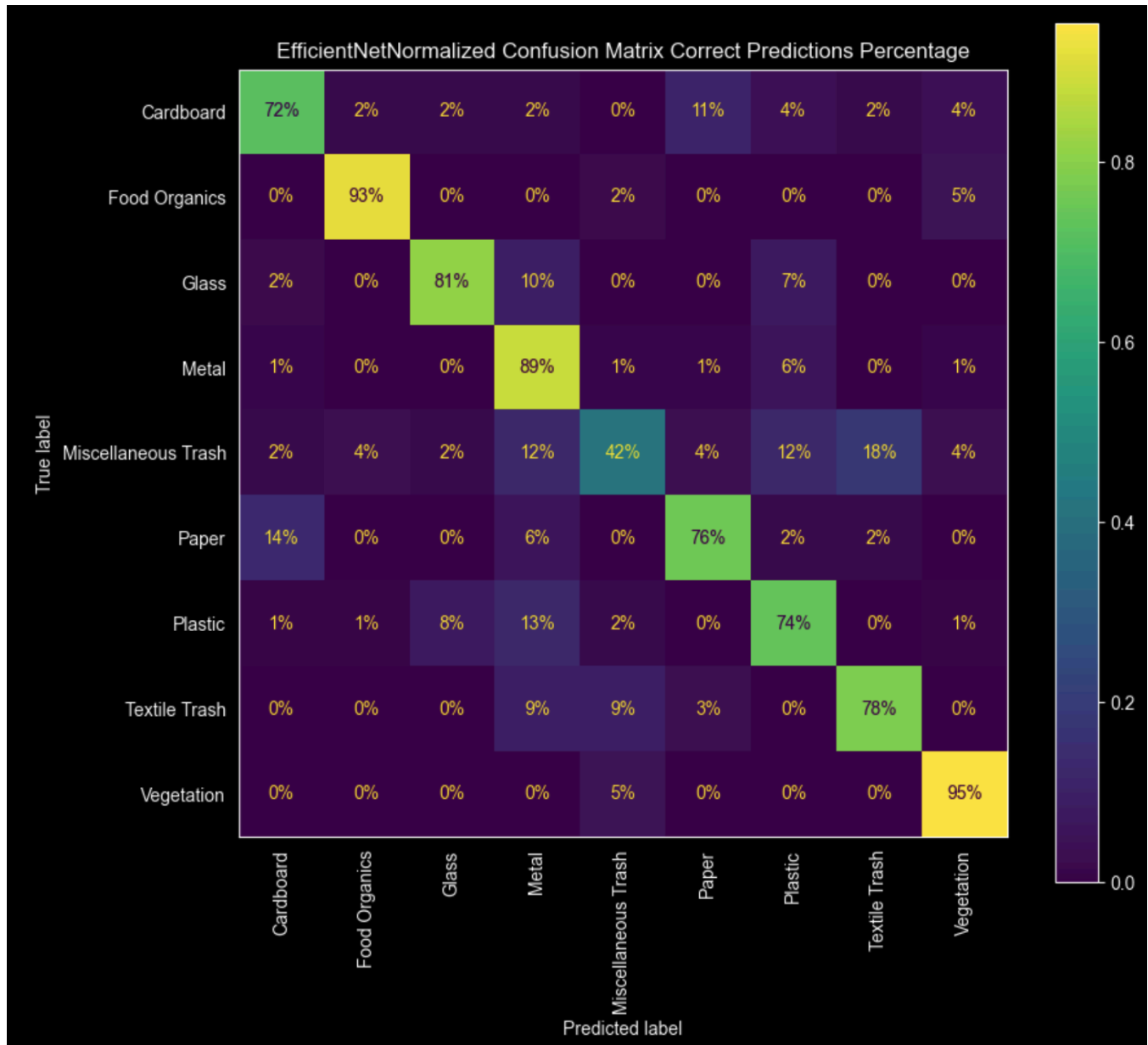
Note that from the diagram glass is incorrectly classified as plastic approximately 100% of the time, while plastic is incorrectly misclassified as either cardboard or metal. Another interesting error is that textile trash is misclassified as paper. One of the reasons may be that they visually look structurally the same as shown in the sample images above. More research is needed in these areas to determine why these large misclassifications occur.

For a final numeric chart based on our MobileNet model, we have a performance table which measures precision and recall. Recall can be defined as the percentage of correct classifications that exist while precision can be defined as how close we are to our actual prediction. Below is a chart illustrating this:

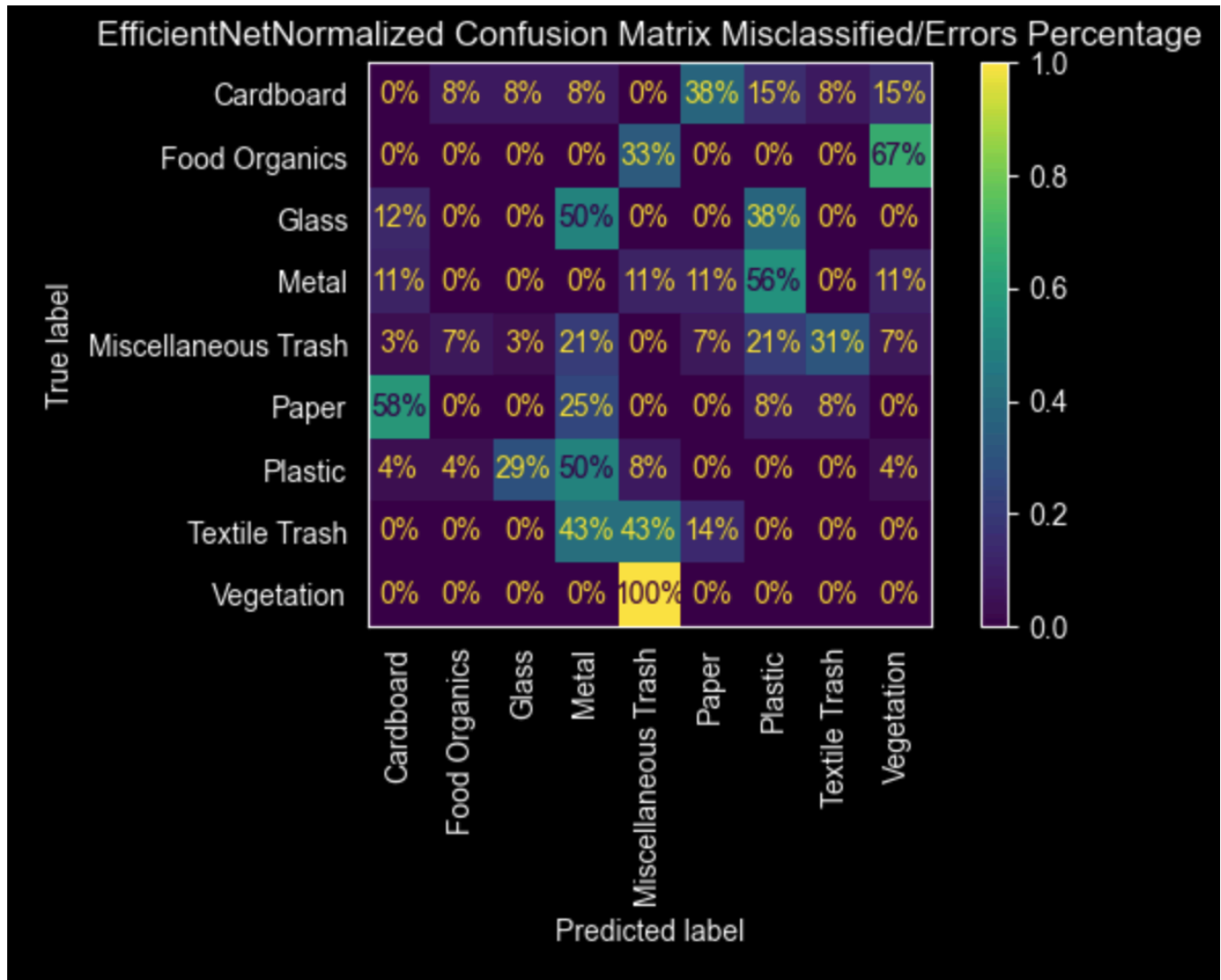


EfficientNetB0 Results

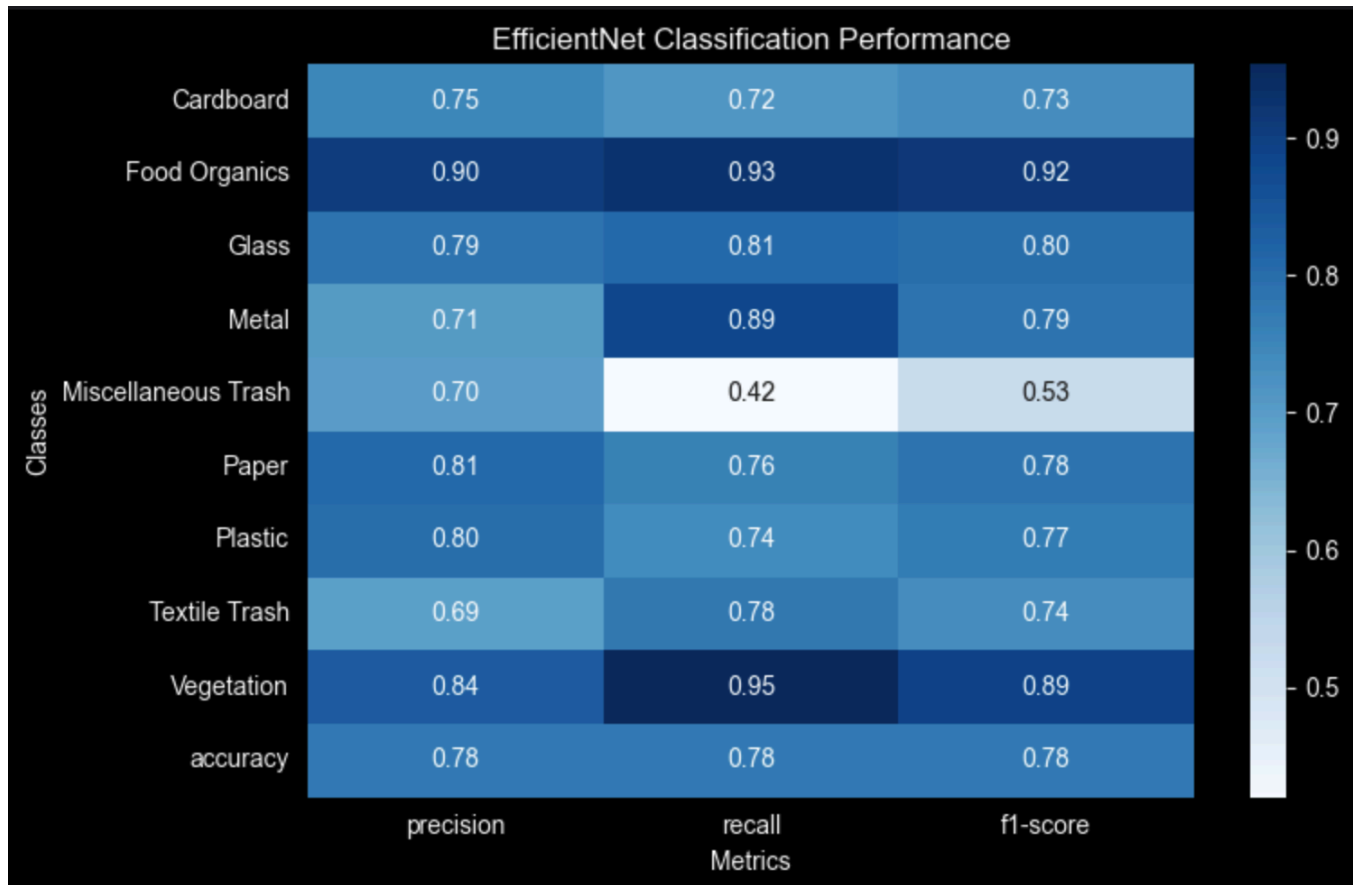
Similarly, we generated normalized confusion matrices and performance charts for the EfficientNetB0 and GoogLeNet models to compare classification behavior across different models. Both models demonstrated strong performance overall, with most correct predictions concentrated along the diagonal of the confusion matrices. Similar patterns of confusion occurred between visually overlapping categories such as Plastic, Glass, Metal, and Miscellaneous Trash. Continued on the next pages are these matrices and charts:



As shown by the graph above, EfficientNetB0 had the lowest prediction accuracy rates, especially misclassifying Miscellaneous Trash. Also note that metal seems to have a high bias towards metal, shown by the metal column highlighted that is not a diagonal cell which coincides with the imbalance data as shown above in the data preparation section. Another bias that we can see within the matrix is towards the plastic label. Plastic was the label with the most instances in our dataset.

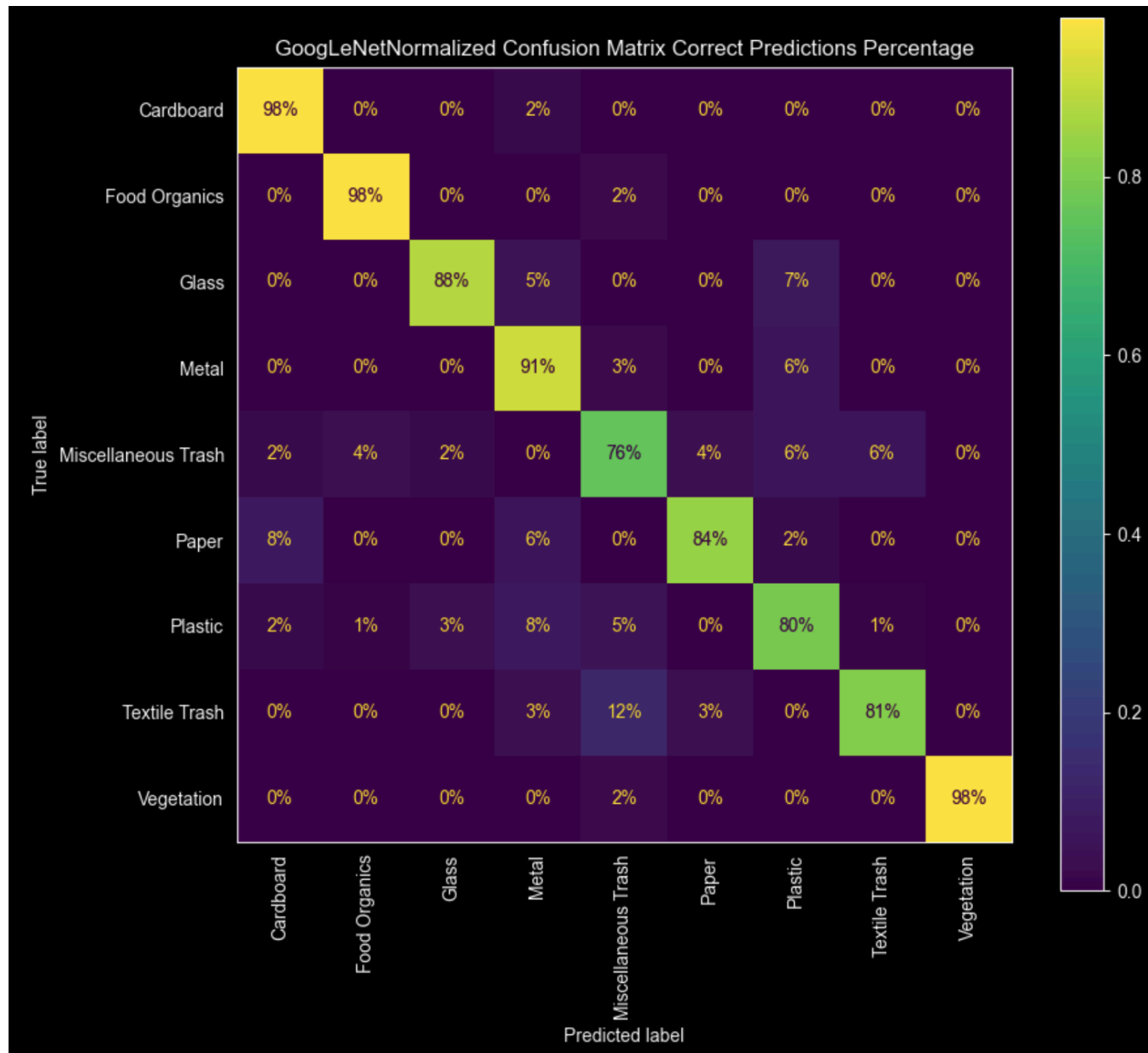


We can see the errors more in detail in the image above for EfficientNet. Vegetation is misclassified as miscellaneous trash 100% of the time. This could indicate that when the model encounters vegetation and is not certain, it will classify it as miscellaneous trash where in human eyes this could also be confusion as demonstrated by the image labels above. Note that Miscellaneous and Cardboard are misclassified across the spectrum which correlates with overall prediction of the two labels with the lowest prediction rates. Overall this model's error highlights the poor prediction compared to the other models as the heat map is all over the place. This suggests that the model fails to gather the concept of the classes: Cardboard, Metal, Paper, and Miscellaneous Trash since it misclassifies them across many classes.

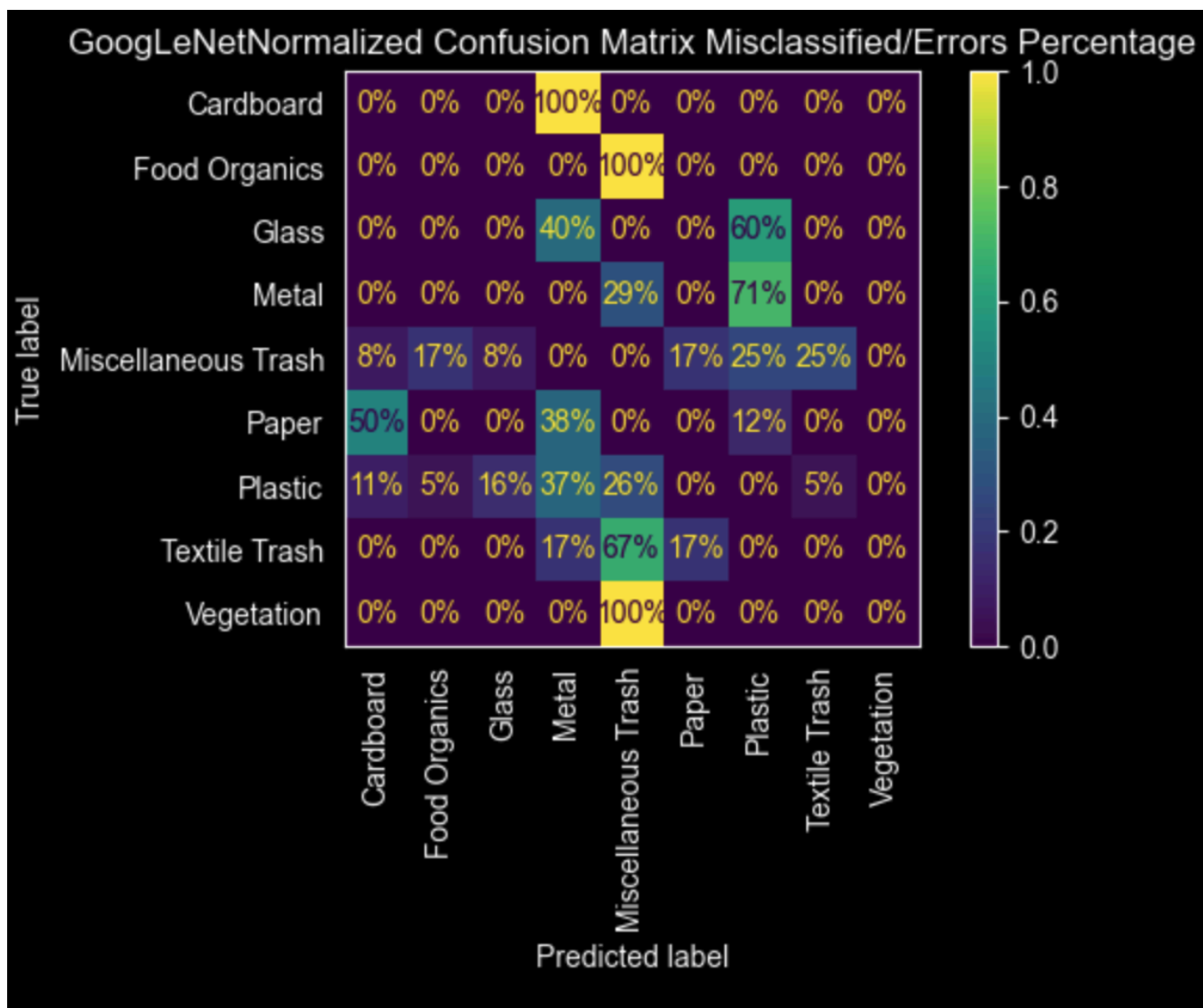


This model's poor performance is emphasized in the precision, recall, and f1-score metrics compared to all models. This is due to the poor performance in prediction Miscellaneous Trash. It did not manage to clearly grasp what makes an object a Miscellaneous Trash.

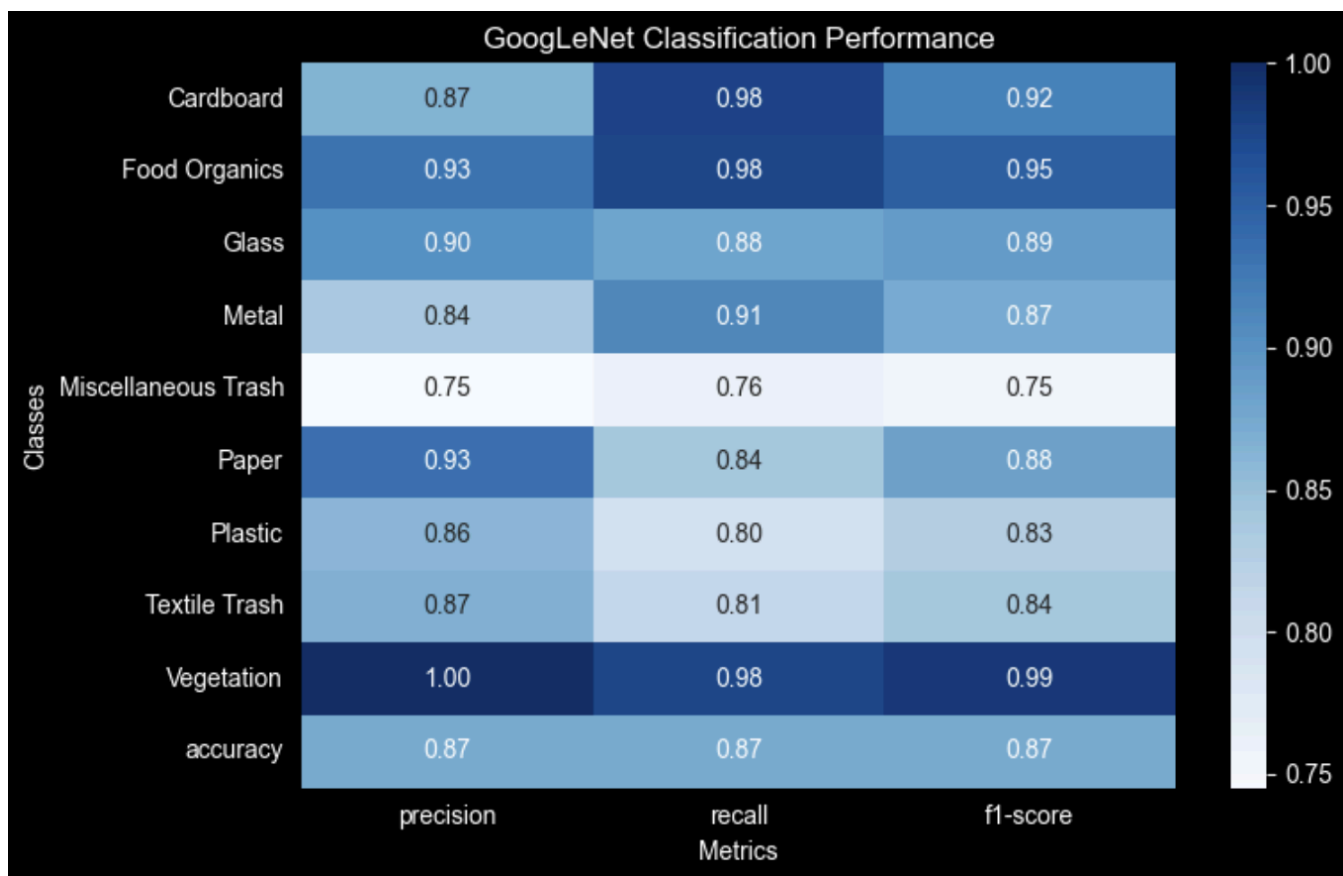
GoogleNet Result



GoogleNet overall gave the best prediction of the non-yolo models. It suffered from the same misclassification of Miscellaneous Trash as the other models but had a higher prediction rate. As with the other models, it suffered from the imbalance dataset by having a strong bias towards the Metal and Plastic labels shown in the columns.

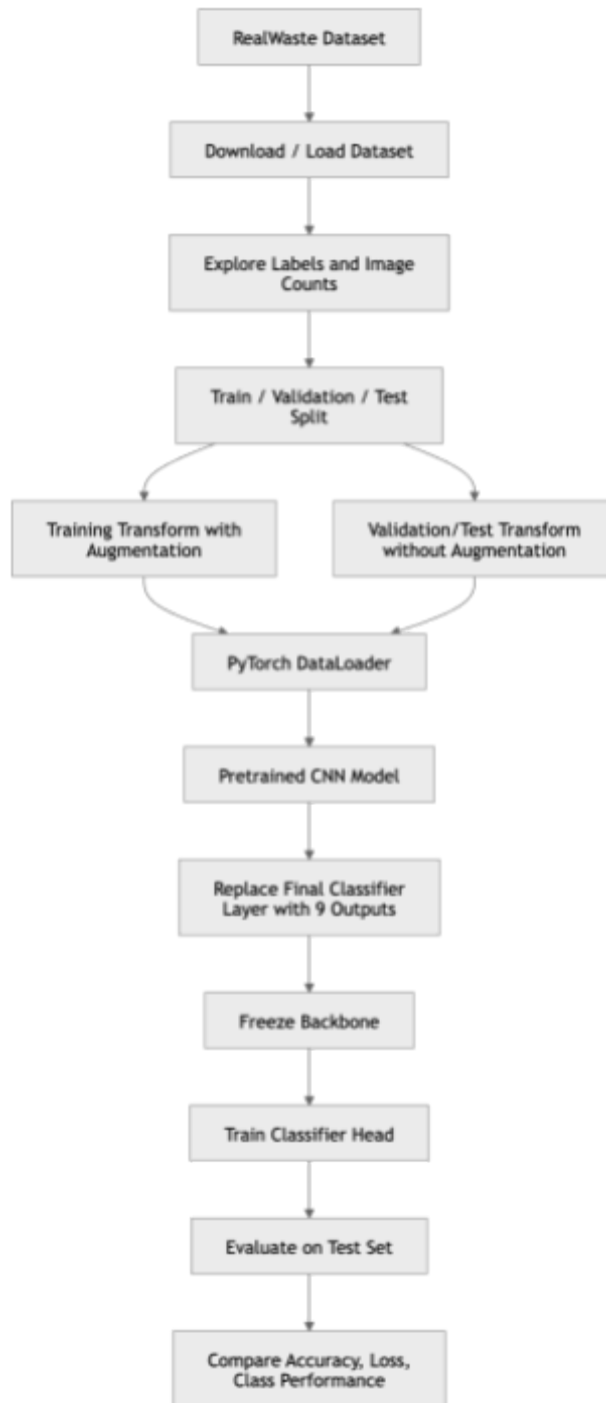


Above we can see that even though GoogLeNet still had a higher prediction accuracy, it still managed to misclassify Miscellaneous Trash. This model also had a bias towards plastic and metal. The model's poor performance was due to not having a concrete concept of the class plastic as it was also misclassified as many labels.



Overall, GoogLeNet showed the strongest performance out of all three of the different models, with an overall test accuracy of 87%. This model did take substantially longer to train than MobileNet. If we also take into account the training time, the MobileNet model did the best job at optimizing both the test accuracy and the training time. MobileNet also had less confusion as many of the labels were misclassified by one or two labels. It made nearly the same accurate predictions in the least amount of training time compared to all of the other models.

On the next page is a detailed diagram of the training procedure of our models:



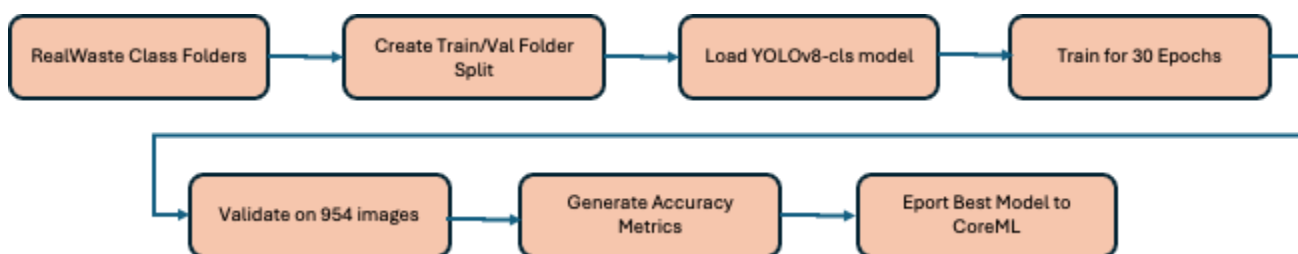
To summarize our approach, a diagram is shown above. To go in detail from this diagram, first we start with our RealWaste Dataset and then proceed to download and load the dataset using code in the Jupyter Notebook. Next, we explore labels and image counts and display them for each category. From there, we split the data into three groups using a train, validation, and test split that is a stratified split. We split into two subgroups: a training transform with data augmentation and a validation and test transform without data augmentation. Next,

we use PyTorch's DataLoader and various pretrained CNN Models. After that, we replace the final classifier layer with 9 outputs. We start out configuring the parameters to train the outer layer. We start with a large number of 0.01 or 0.001 learning rate and a small epoch to see which hyper-parameters give us a good result. After determining which learning rate gave us the best training and validation loss, we committed to the configuration and moved to the next upper layers. After finalizing the training for the first outer layer we then gradually unfreeze the top layers until no further improvements are seen. We did this by decreasing the learning rate since the model was mostly optimized to fit our dataset in order to not trigger an exploding gradient. We also experimented with other parameters such as the optimizer and the learning scheduler. We found that Adam W with weight decay was the best optimizer and Cosine Annealing for the best learning scheduler for our models gave us the best result to reduce the overall model's error. One of the reasons for this, was that since we are using pretrained models, we are already close to the optimal weights since the pretrained has already been trained to recognize overall structure features but just needed to focus on those features to recognize our labels in our dataset. Therefore we needed optimizers, learning schedulers, and a learning rate that did not contribute to large momentum that diverges away from the desired weights to reduce the model's error. Finally, after reducing the model's error by fine tuning, we saved the model to be compared and analyzed.

YOLOv8 Classification Notebook

Now, we examine our next model, YOLOv8 which is used as a classification [3] model, rather than object detection. Below is a diagram that portrays the underlying structure of our model.

Implementation diagram



First, we begin with RealWaste Class Folders, and then we create a training, validation, and test split using code in the Jupyter Notebook. Next, we load the YOLOv8-cls model, train for 30 Epochs, validate on 954 images, generate accuracy metrics, and export the best model to test. To improve performance from our baseline mode, data augmentation and a stratified rather than random split was used. The dataset is reorganized into YOLO classification format[4] with separate train and val folders. Training used the pretrained YOLO model [yolov8s-cls.pt](#), an image size of 224, a batch size of 16, and Apple Silicon MPS acceleration when available. The training set contained 3,798 images, and the validation set contained 954 images.

Our baseline model was performed without data augmentation and using random split. The code to train the baseline model is shown on the next page:

```
model.train(  
    ### data="realwaste-yolo/RealWaste/data.yaml",  
    ### '/content/RealWaste/data.yaml'  
    data="outputs_for_YOLO_classification_baseline",  
    epochs=10, ###==> Hyper-parameters  
    lr0=0.005,  
    lrf=0.1,  
    momentum=0.9,  
    weight_decay=0.0001,  
    ###warmup_epochs=5,  
    ###warmup_momentum=0.8,  
    ###warmup_bias_lr=0.1, ###<==  
    imgsz=224,  
    batch=16,  
    patience=25,  
    device=device,  
    workers=0, # safer in Jupyter if needed  
    project="runs",  
    name="yolov8n_cls_baseline_10epochs"  
)
```

The test accuracy for the baseline model was 67%. The results from this run are shown below:

Top-1 Accuracy: 0.6704
Top-5 Accuracy: 0.9874

The first enhancement included to increase test accuracy was substituting the default random split to use a stratified split. To accomplish this task, the following code for a random split on the next page:

```

train_paths, temp_paths, train_targets, temp_targets = train_test_split(
    image_paths,
    targets,
    train_size=train_ratio,
    random_state=seed
)

val_paths, test_paths, val_targets, test_targets = train_test_split(
    temp_paths,
    temp_targets,
    test_size=0.5,
    random_state=seed
)

```

Was replaced with the following code:

```

train_paths, temp_paths, train_targets, temp_targets = train_test_split(
    image_paths,
    targets,
    train_size=train_ratio,
    stratify=targets,
    random_state=seed
)

val_paths, test_paths, val_targets, test_targets = train_test_split(
    temp_paths,
    temp_targets,
    test_size=0.5,
    stratify=temp_targets,
    random_state=seed
)

```

Notice that for the stratified split, all we needed to add was the stratify parameter in both instances of code.

In implementing this change alone, test accuracy increased dramatically, from 67% to 88%. Our results are shown here:

```

Top-1 Accuracy: 0.8836
Top-5 Accuracy: 0.9986

```

This suggests that due to the unbalanced nature of our dataset categories, stratifying the data was a key solution to this problem in our data. Stratifying data maintains key proportions within each category rather than maintaining the same proportion of data in each category that occurs with a random split.

The next modification to our model was to add in data augmentation from the stratified dataset. In our training of the YOLO model, the following parameters were added as shown below:

```
model.train(  
    ### data="realwaste-yolo/RealWaste/data.yaml",  
    ### '/content/RealWaste/data.yaml'  
    data="outputs_for_YOLO_classification_stratified_and_data_augmented_dataset",  
    epochs=10, ###==> Hyper-parameters  
    lr0=0.005,  
    lrf=0.1,  
    momentum=0.9,  
    weight_decay=0.0001,  
    ###warmup_epochs=5,  
    ###warmup_momentum=0.8,  
    ###warmup_bias_lr=0.1, ###<==  
    imgsz=224,  
    batch=16,  
    patience=25,  
    device=device,  
    workers=0, # safer in Jupyter if needed  
    project="runs",  
    name="yolov8n_cls_stratified_aug_10epochs",  
    degrees=8,  
    scale=0.15,  
    fliplr=0.5,  
    hsv_h=0.01,  
    hsv_s=0.05,  
    hsv_v=0.05,  
    perspective=0.0,  
)
```

The parameters added were the degrees which signifies a rotation and the scale parameter was chosen for random zooming in and out of the image to change the size of the test images. The `fliplr` parameter represents the horizontal flip probability, `hsv_h` stands for the hue, `hsv_s` for saturation, and `hsv_v` for brightness and value. Together, the last three parameters perform very subtle color changes and modify the data just enough so that the model can learn more thoroughly. Setting the last parameter, `perspective`, equal to 0.0 ensures that no perspective warping occurs.

In adding data augmentation to the already stratified dataset, we saw a small change in test accuracy. The model increased from an 88% test accuracy to 90% test accuracy. Our results are shown below:

Top-1 Accuracy: 0.9018
Top-5 Accuracy: 0.9972

This was the highest accuracy in total of all of our adjustments suggesting that data augmentation did have an impact on improving the model overall, even though that impact was smaller in comparison to stratifying the dataset alone.

Note that increasing the number of epochs to 100 with our best model did not improve test accuracy suggesting that overfitting may have happened sooner with YOLO classification than with other models.

Due to our results, this model appears to be one of the strongest classification results among the notebooks.

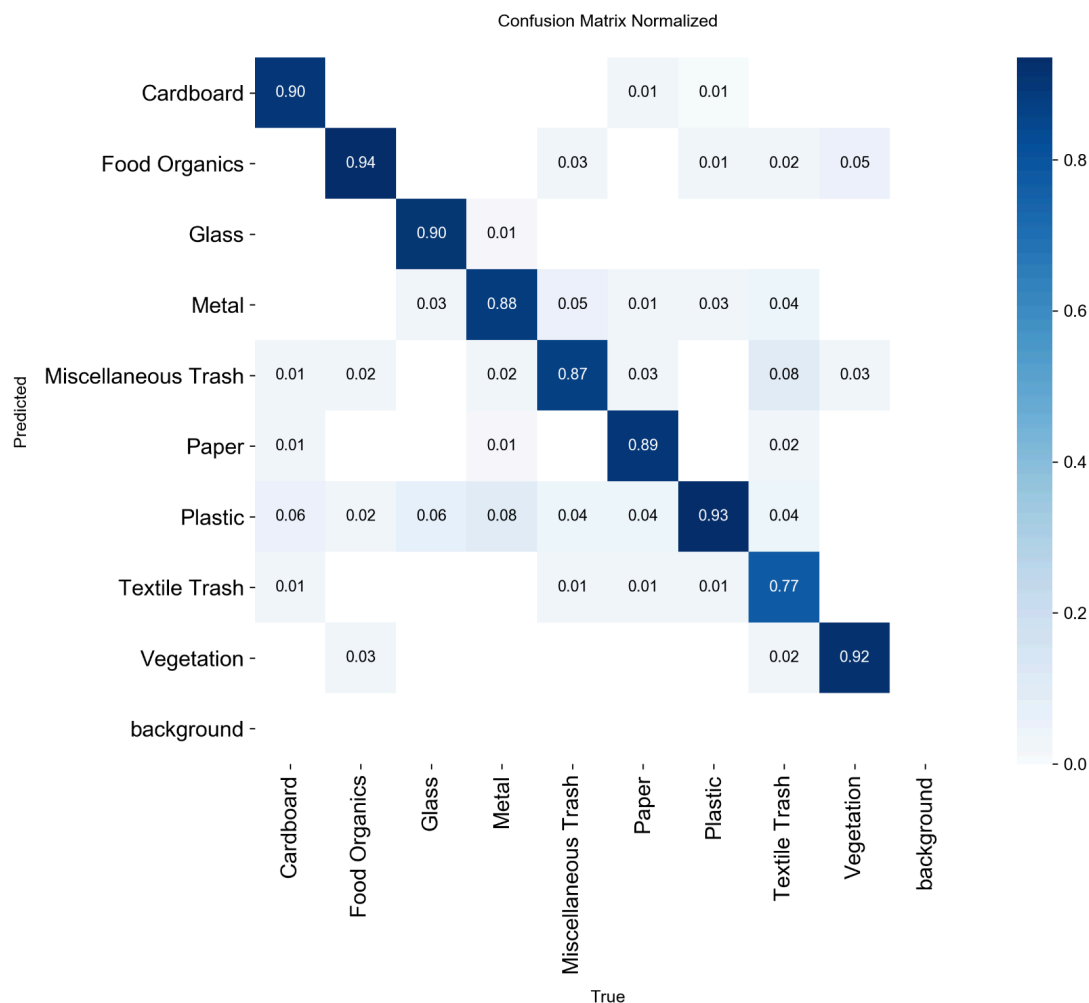
Configuration hyper-parameters after freeze backbone and add unfreeze additional top layers after train classifier head.

```
### Clasify model.
model = YOLO("yolov8n-cls.pt")

model.train(
    ### data="realwaste-yolo/RealWaste/data.yaml",
    ### '/content/RealWaste/data.yaml'
    data="outputs_for_YOLO_classification_stratified_dataset",
    epochs=60, ###==> Hyper-parameters
    freeze=8,
    lr0=1e-4,
    lrf=0.01,
    momentum=0.9,
    weight_decay=5e-4,
    dropout=0.2,
    # Augmentation
    hsv_h=0.015,
    hsv_s=0.7,
    hsv_v=0.4,
    degrees=10.0,
    translate=0.1,
    scale=0.3,
    flipplr=0.5,
    imgsz=224,
    batch=64,
    patience=15,
    optimizer="AdamW",
    label_smoothing=0.1,
    device=device,
    workers=5, # safer in Jupyter if needed
    pretrained=True,
    verbose=True,
    project="runs",
    name="yolov8n_cls_stratified_10epochs"
)
```

Freeze the backbone, learning ratio reduced to 1e-4 and 0.01, include a dropout = 0.2 and AdamW as optimizer. The training was set to run for 60 epochs but it stopped early since the model was not improved in the last 15 attempts (patience = 15). Nevertheless, the model reached high accuracy, but not enough to pass the best model.

Top-1 Accuracy: 0.8822
Top-5 Accuracy: 0.9986

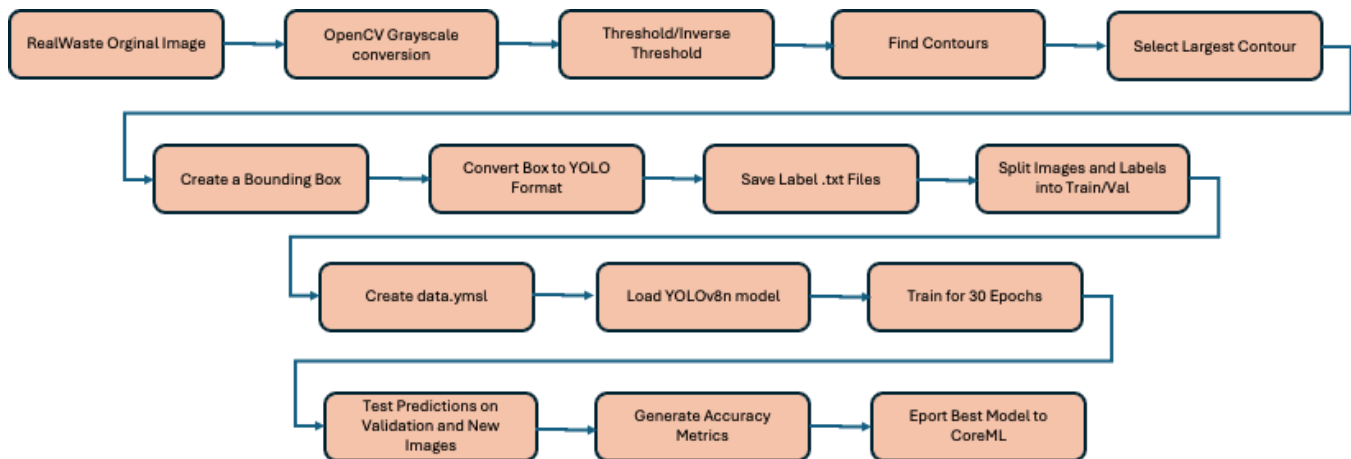


YOLO Detection Notebook

Here, we explore a variation of the mode, a YOLO detection notebook capable of detecting multiple objects in the same photo. Our main goal here was to convert RealWaste into an object detection dataset by generating bounding boxes automatically, then training the model using YOLOv8 detection.

Below is an implementation diagram of the architecture of our model.

Implementation diagram



First, we start with the RealWaste original image, which is the starting image from the dataset. Next, we perform an OpenCV grayscale conversion which is the original image converted from color to a black and white grayscale. The purpose of this conversion is that it makes it easier for the computer to separate the object from the background. Then, we perform a threshold and inverse threshold where the code turns the grayscale image into mostly black and white pixels. The purpose of the preprocessing task is to try to separate the object we are classifying from its background. The following code achieves these tasks:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, thresh_inv = cv2.threshold(
    gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)
```

After that, we find contours which are basically the outline of a visible shape to determine where the major shapes/blobs are in this image. The following code below demonstrates this:

```
contours, _ = cv2.findContours(
    thresh_inv, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)
```

Then, we select the largest contour where the notebook assumes the biggest detected shape is most likely the waste object using the following code:

```
largest = max(contours, key=cv2.contourArea)
x, y, box_w, box_h = cv2.boundingRect(largest)
```

Next, we create a bounding box where the code draws a rectangle around the largest contour. This rectangle becomes the estimated object location. This step is essential so that classification data is transformed into detection data. After, we convert the box into a YOLO format. Yolo needs labels such as this:

“class_id center_x center_y width height”

All values are normalized between 0 and 1, meaning we will get some sample output such as

“Class 6, object centered around the middle of the image, with that width and height”

Then, we save labels as .txt files so that for each image, the notebook creates a matching .txt file. The .txt file contains the bounding box label that YOLO needs. After saving the labels, we split images into training and validation folders. This is because YOLO detection expects a structure such as:

```
RealWaste/  
├── images/  
│   ├── train/  
│   └── val/  
├── labels/  
│   ├── train/  
│   └── val/  
└── data.yaml
```

After splitting images, we create a data.yaml file. This is a structured file that tells YAML where the train and validation images are located, how many classes there are, and what the class names are. Next, we load the YOLOv8n model which uses the small YOLOv8 detection model called “[yolo8n.pt](#)” using the following code:

```
model = YOLO("yolov8n.pt")
```

Note that this model is for detection YOLO not classification YOLO so multiple objects can be detected at the same time. After that, we train for 30 epochs in which the model trains on the generated bounding boxes using the code below:

```
model.train(  
    data="/path/to/data.yaml",  
    epochs=30,  
    imgsz=640,  
    batch=8,  
    device="mps"  
)
```

Note that the boxes are only automatically estimated, so the labels may have some small error. The next step of our model is to test predictions on validation and new images. After training, the notebook tests to determine whether YOLO can draw boxes around waste objects. Finally, we generate accuracy metrics and export the best model to CoreML which allows the model to be used in applications designed for mobile devices such as the iPhone.

Initial training accidentally used CPU, then was corrected to use device = “mps” to reduce training time. Detection was tested on one plastic image, full plastic folder, glass folder, validation images, and new real-world test images. The notebook found an important limitation: the model often detects one large object

box, not multiple separate waste objects. This is expected because the training labels were automatically generated as one bounding box per image. Therefore, this YOLO detection version is useful as a prototype, but not yet a full multi-object detector.

We implemented the following training using Google Colab to use a CUDA GPU and it served as a baseline for our other experiments. The hyperparameters we used were 30 epochs, an image size of 640, and a batch size of 8. Note that we used a smaller batch size to speed up training since larger images require more GPU memory. We trained the model using this code below:

```
model.train(
    ### data="realwaste-yolo/RealWaste/data.yaml",
    ### '/content/RealWaste/data.yaml'
    data="/content/drive/MyDrive/Colab Notebooks/RealWaste/data.yaml",
    epochs=30,
    imgsz=640,
    batch=8,
    device=device,
    workers=0, # safer in Jupyter if needed
    project="runs",
    name="yolov8_pytorch_first"
)
```

This experiment produced the following outcome, and served as a baseline YOLOv8 model. The test accuracy for the metric mAP50-95 for all categories is highlighted in yellow and is found to be 72.7%. Note that training took a total of 1 hour 21 minutes and 28 seconds. The output of our experiment is shown below:

```
Validating /content/runs/detect/runs/yolov8_pytorch_first/weights/best.pt...
Ultralytics 8.4.46 🚀 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 3,007,403 parameters, 0 gradients, 8.1 GFLOPs
```

	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
60/60		3.7it/s	16.2s				
	all	954	954	0.872	0.823	0.909	0.727
	Cardboard	93	93	0.877	0.882	0.929	0.671
	Food Organics	83	83	0.818	0.807	0.88	0.651
	Glass	84	84	0.931	0.802	0.951	0.747
	Metal	158	158	0.89	0.816	0.93	0.765
	Miscellaneous Trash	99	99	0.867	0.595	0.787	0.671
	Paper	100	100	0.92	0.806	0.921	0.751
	Plastic	185	185	0.885	0.828	0.916	0.674
	Textile Trash	64	64	0.727	0.914	0.887	0.778
	Vegetation	88	88	0.933	0.955	0.981	0.831

```
Speed: 0.2ms preprocess, 3.1ms inference, 0.0ms loss, 0.9ms postprocess per image
Results saved to /content/runs/detect/runs/yolov8_pytorch_first
Training ended at: 2026-05-04 04:46:28
Elapsed time: 1h 21m 28s
```

Our second experiment with our model is shown with the new hyperparameters added - a new learning rate, momentum, weight decay, warmup epochs, warmup momentum, a warmup bias learning rate, a new image size of 224 to speed up training and a batch size of 16. Because the image size was smaller, the batch size could be increased from 8 to 16.

```
model.train(
    ### data="realwaste-yolo/RealWaste/data.yaml",
    ### '/content/RealWaste/data.yaml'
    data="/content/drive/MyDrive/Colab Notebooks/RealWaste/data.yaml",
    epochs=30, ###==> Hyper-parameters
    lr0=0.005,
    lrf=0.1,
    momentum=0.9,
    weight_decay=0.0001,
    warmup_epochs=5,
    warmup_momentum=0.8,
    warmup_bias_lr=0.1, ###<==
    imgsz=224,
    batch=16,
    device=device,
    workers=0, # safer in Jupyter if needed
    project="runs",
    name="yolov8_pytorch_first"
)
```

This experiment produced the following outcome. The test accuracy for all categories for the metric mAP50-95 is highlighted in yellow and is found to be 74.5%, which is a 1.8 percentage point increase from the previous experiment. Note that training took a total of 47 minutes and 5 seconds, which decreased from the initial training time of 1 hour and 21 minutes. This was due to decreasing the size of the image from 640 to 224. The output of our experiment is shown below:

30 epochs completed in 0.767 hours.

Optimizer stripped from /content/runs/detect/runs/yolov8_pytorch_first/weights/last.pt, 6.2MB

Optimizer stripped from /content/runs/detect/runs/yolov8_pytorch_first/weights/best.pt, 6.2MB

Validating /content/runs/detect/runs/yolov8_pytorch_first/weights/best.pt...

Ultralytics 8.4.46 🚀 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

Model summary (fused): 73 layers, 3,007,403 parameters, 0 gradients, 8.1 GFLOPs

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%
30/30	2.7it/s	11.1s				
all	954	954	0.855	0.823	0.901	0.745
Cardboard	93	93	0.837	0.882	0.909	0.744
Food Organics	83	83	0.828	0.871	0.922	0.704

Glass	84	84	0.934	0.786	0.913	0.75
Metal	158	158	0.804	0.81	0.877	0.712
Miscellaneous Trash	99	99	0.787	0.677	0.83	0.716
Paper	100	100	0.929	0.785	0.904	0.787
Plastic	185	185	0.887	0.803	0.921	0.711
Textile Trash	64	64	0.752	0.812	0.843	0.725
Vegetation	88	88	0.938	0.977	0.985	0.856

Speed: 0.0ms preprocess, 0.5ms inference, 0.0ms loss, 0.9ms postprocess per image
 Results saved to **/content/runs/detect/runs/yolov8_pytorch_first**
 Training ended at: 2026-05-05 00:49:36
 Elapsed time: 0h 47m 5s

One reason performance improved was a combination of better optimization settings. For example, learning rate tuning helped the model learn more effectively. The learning rate determines how large the model's weight updates are during training. A better learning rate can help the model converge more smoothly. The final learning rate factor of $\text{lrf} = 0.1$ gradually reduces the learning rate toward the end of training which helps the model make larger updates earlier and reserves smaller and more meticulous updates later in the training process.

The next change that helped the experiment improve was momentum which served to stabilize training. We set the momentum parameter to 0.9 which helps the optimizer keep moving in a useful direction instead of reacting too strongly to noisy batches. This functions to make training smoother and helps the model reach a more accurate solution. It is especially useful when training on image datasets where there is substantial variation between batches.

Also, weight decay helped reduce overfitting as previously used with regular CNN architectures. Weight decay is a regularization technique. We set weight decay to 0.0001 which discouraged the model from relying too heavily on overly specific features in the training data. This can improve generalization on validation and test images. Since we were using the RealWaste dataset, this has an impact because the models need to recognize material categories rather than memorize exact data. As a result, this prevents overfitting.

Next, using warmup parameters helped training start more safely. We used the following parameters to achieve this goal: `warmup_epochs`, `warmup_momentum`, and `warmup_bias`, and set these equal to 5, .8, and .1 respectively. Warmup gradually adapts training behavior during the first few epochs which prevents the model from making unstable updates too early. This is particularly useful when working with transfer learning because prechanged weights should not be changed too aggressively at the beginning. The result of this change is that the model was able to fine-tune more smoothly.

Using a smaller image size made training faster. Reducing the image size from 640 to 224 greatly decreased the amount of computation per image. In turn, training time reduced from an initial 1 hour and 21 minutes in the previous run to a mere 47 minutes. A possible downside of this change is that smaller images can sometimes reduce detail. However, in our case, the model still improved which suggests that tuning the hyperparameters compensated for a decrease in image size.

Finally, larger batch size may have improved training stability. The smaller image size allowed the batch size to increase from 8 to 16. A larger batch size gives the optimizer a more stable estimate of the training direction which reduces unstable updates and results in an overall improvement in final performance.

To avoid exhausting GPU resources, warmup was removed and we trained with CPU only. The next significant change in this run was increasing the total number of epochs to 100. The other parameter added was setting patience to 25. This means that if the model does not improve in validation performance after 25 epochs, training is stopped.

```
model.train(  
    ### data="realwaste-yolo/RealWaste/data.yaml",  
    ### '/content/RealWaste/data.yaml'  
    data="/content/drive/MyDrive/Colab Notebooks/RealWaste/data.yaml",  
    epochs=100, ###==> Hyper-parameters  
    lr0=0.005,  
    lrf=0.1,  
    momentum=0.9,  
    weight_decay=0.0001,  
    ###warmup_epochs=5,  
    ###warmup_momentum=0.8,  
    ###warmup_bias_lr=0.1, ###<==  
    imgsz=640,  
    batch=16,  
    patience=25,  
    device=device,  
    workers=0, # safer in Jupyter if needed  
    project="runs",  
    name="yolov8_pytorch_first"  
)
```

The results of this model are shown below:

100 epochs completed in 27.210 hours.

Optimizer stripped from /Users/car/CSUChico/CSCI611/yolov8/runs/detect/runs/yolov8_pytorch_first-9/weights/last.pt,
6.3MB

Optimizer stripped from /Users/car/CSUChico/CSCI611/yolov8/runs/detect/runs/yolov8_pytorch_first-9/weights/best.pt,
6.3MB

Validating /Users/car/CSUChico/CSCI611/yolov8/runs/detect/runs/yolov8_pytorch_first-9/weights/best.pt...

Ultralytics 8.4.41 🚀 Python-3.10.20 torch-2.11.0 MPS (Apple M5)

Model summary (fused): 73 layers, 3,007,403 parameters, 0 gradients, 8.1 GFLOPs

WARNING ⚠️ NMS time limit 2.050s exceeded

Class Images Instances Box(P R mAP50 mAP50-95): 100% 30/30
 16.4s/it 8:12.6ss31

all	954	954	0.892	0.886	0.932	0.766
Cardboard	93	93	0.856	0.893	0.914	0.713
Food Organics	83	83	0.9	0.871	0.926	0.717
Glass	84	84	0.929	0.928	0.98	0.788
Metal	158	158	0.9	0.857	0.932	0.778
Miscellaneous Trash	99	99	0.865	0.818	0.863	0.745
Paper	100	100	0.894	0.82	0.927	0.793
Plastic	185	185	0.881	0.892	0.925	0.713
Textile Trash	64	64	0.857	0.906	0.929	0.803
Vegetation	88	88	0.947	0.989	0.99	0.846

Speed: 2.4ms preprocess, 350.0ms inference, 0.0ms loss, 6.3ms postprocess per image

Results saved to /Users/car/CSUChico/CSCI611/yolov8/runs/detect/runs/yolov8_pytorch_first-9

Training ended at: 2026-05-06 17:36:25

Elapsed time: 27h 25m 59s

This model reached the highest test accuracy for the mAP50 metric at 76.6%. The reason for this change was increasing the total number of epochs to 100. However, a significant downside to using this approach, is that training time increased significantly taking a total of 27 hours and 25 minutes to train the model.

Other methods were used to improve the performance of the model such as data augmentation. These methods were found to have little impact on model performance compared to increasing the number of epochs.

The code to implement data augmentation is shown highlighted below:

```
model.train(
  data="realwaste-yolo/RealWaste/data.yaml",
  ### '/content/RealWaste/data.yaml'
  ### data="/content/drive/MyDrive/Colab Notebooks/RealWaste/data.yaml",
  epochs=10, ###==> Hyper-parameters less epoch because the training take a long time.
  #lr0=0.005, for now just focus on data argumentation.
  #lrf=0.1,
  # =====
  # augmentation settings
  # =====
  degrees=15.0, # rotation
  flipud=0.1, # vertical flip probability
  fliplr=0.5, # horizontal flip probability
  hsv_h=0.015, # hue
  hsv_s=0.7, # saturation
  hsv_v=0.4, # brightness/value
  translate=0.1,
  scale=0.5,
  shear=2.0,
```

```

mosaic=1.0,
mixup=0.1,
# =====
momentum=0.9,
weight_decay=0.0001,
###warmup_epochs=5,
###warmup_momentum=0.8,
###warmup_bias_lr=0.1,
imgsz=640,
batch=32,      # use more memory but improve performance
patience=5,    # if the accuracy does not improve after 5 epochs then stop the training.
device=device,
workers=5,      # make it faster in M5 Macbook -- safer in Jupyter if needed ###<==
project="runs",
name="yolov8_pytorch_first"
)

```

The output of this run is shown here.

Training started at: 2026-05-06 20:47:09

I am training on: MPS

10 epochs completed in 0.619 hours.

Optimizer stripped from /Users/car/CSUChico/CSCI611/yolov8/runs/detect/runs/yolov8_pytorch_first-10/weights/last.pt, 6.2MB

Optimizer stripped from /Users/car/CSUChico/CSCI611/yolov8/runs/detect/runs/yolov8_pytorch_first-10/weights/best.pt, 6.2MB

Validating /Users/car/CSUChico/CSCI611/yolov8/runs/detect/runs/yolov8_pytorch_first-10/weights/best.pt...

Ultralytics 8.4.41  Python-3.10.20 torch-2.11.0 MPS (Apple M5)

Model summary (fused): 73 layers, 3,007,403 parameters, 0 gradients, 8.1 GFLOPs

Class Images Instances Box(P R mAP50 mAP50-95): 100% 15/15
1.5s/it 22.1s1.2s

all	954	954	0.771	0.752	0.842	0.578
Cardboard	93	93	0.684	0.86	0.865	0.562
Food Organics	83	83	0.765	0.831	0.882	0.573
Glass	84	84	0.721	0.75	0.816	0.566
Metal	158	158	0.837	0.713	0.863	0.562
Miscellaneous Trash	99	99	0.658	0.535	0.653	0.482
Paper	100	100	0.867	0.68	0.865	0.606
Plastic	185	185	0.791	0.778	0.845	0.542
Textile Trash	64	64	0.752	0.664	0.816	0.591
Vegetation	88	88	0.868	0.955	0.972	0.715

Speed: 0.1ms preprocess, 4.9ms inference, 0.0ms loss, 4.1ms postprocess per image

Results saved to /Users/car/CSUChico/CSCI611/yolov8/runs/detect/runs/yolov8_pytorch_first-10

Training ended at: 2026-05-06 21:24:52

Elapsed time: 0h 37m 42s

This resulted in a test accuracy mAP50 metric of .578 which meant this did not increase the model performance.

Increasing the total epochs to 30 and still using data augmentation, we obtain the following output:

Ultralytics 8.4.41 🚀 Python-3.10.20 torch-2.11.0 MPS (Apple M5)

Model summary (fused): 73 layers, 3,007,403 parameters, 0 gradients, 8.1 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
15/15 1.2s/it 17.6s1.1s						
all	954	954	0.839	0.874	0.916	0.683
Cardboard	93	93	0.838	0.892	0.941	0.658
Food Organics	83	83	0.829	0.879	0.915	0.643
Glass	84	84	0.911	0.857	0.942	0.687
Metal	158	158	0.844	0.886	0.917	0.696
Miscellaneous Trash	99	99	0.789	0.678	0.801	0.613
Paper	100	100	0.878	0.866	0.915	0.704
Plastic	185	185	0.763	0.885	0.899	0.627
Textile Trash	64	64	0.817	0.922	0.926	0.75
Vegetation	88	88	0.883	1	0.99	0.767

Speed: 0.1ms preprocess, 3.7ms inference, 0.0ms loss, 4.2ms postprocess per image

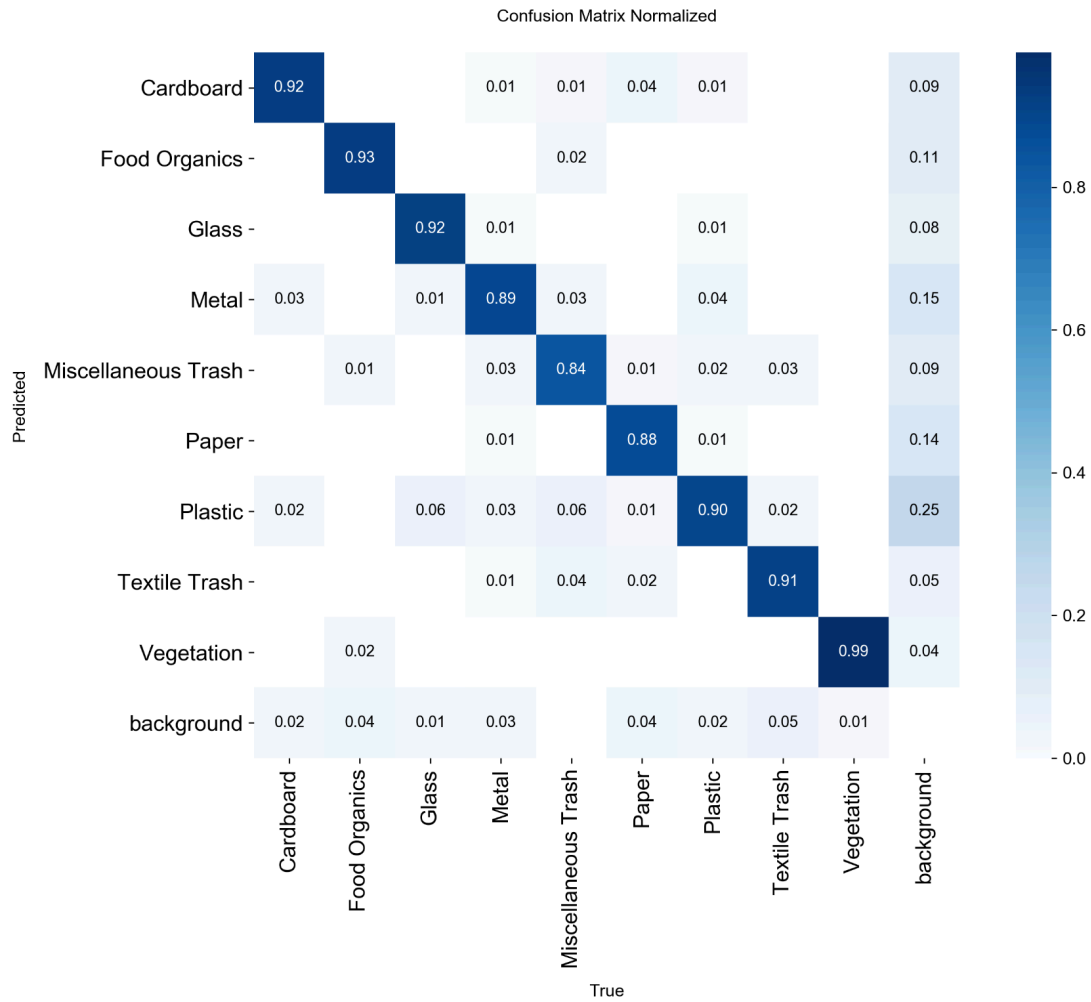
Results saved to /Users/car/CSUChico/CSCI611/yolov8/runs/detect/runs/yolov8_pytorch_first-11

Training ended at: 2026-05-07 00:26:37

Elapsed time: 2h 57m 50s

Using this approach of data augmentation combined with 30 epochs, we obtain a test accuracy for metric mAP50 of 68% which is still a reduction in improvement from our baseline.

To conclude our results, increasing the total number of epochs resulted in significant improvement gains in our model compared to data augmentation which reduced our efficiency in the model.



Comparison

Model Name	Task Type	GFLOPS [10]	Training Time	Top 5 Accuracy	Top 1 Accuracy
YOLOv8n-cls	Classification	12.5	< 1 hour	99.72%	90.18%
YOLOv8n Detection	Object Detection (Box Labels Needed)	28.8	~ 3 hours	N/A	89.2%
GoogleNet	Classification	1.5	< 1 hour	97.79%	87.39%
MobileNetV3	Classification	0.06	< 1 hour	99.37%	86.13%
EfficientB0	Classification	0.39	< 1 hour	99.58%	77.52%

What Worked vs. What Did Not

What Worked:

Some things that were extremely effective in our models include transfer learning, data augmentation, and YOLOv8 classification. Transfer learning worked well due to limited data in our dataset being that we only had under five thousand images total. Moreover, data augmentation provided the variety and foundation that our previous data was lacking. A benefit of data augmentation is that we are able to expand upon our data without adding new images from scratch. We saw the greatest improvement with data augmentation. As for training hyperparameters, we found that AdamW with weight decay and Cosine Annealing for our learning schedule, helps us obtain the minimal model error without having dramatic jumps in validation and training loss. Furthermore, YOLOv8 classification performed very strongly given that YOLO classification matched our dataset initially more than YOLO detection in that our dataset contained one image type per photo and no bounding boxes were included in the initial dataset. Additionally, MPS acceleration improved training time on Apple Silicon which our group used to train the models, whereas previously our training had taken many hours, particularly for YOLO detection as we used a large epoch size of 100 for that model. Next, confusion matrices and accuracy/loss curves allowed us to gain strong report visualizations. Also, COreML export is a good future work path for an iPhone demonstration.

For modern YOLOv8 models, we can improve performance by using workers. This is not a direct hyper-parameter, but it is a parameter to help with performance. To achieve this task, in the MacOS terminal we can run:

```
"sysctl -n hw.ncpu"
```

Which tells us how many cores your local Mac contains. The next step is to use this formula to estimate the number of workers:

Number of workers = (total CPU cores) / 2

For an ideal experiment, the total number of workers is 5. Also, batch size improves performance. It is a good idea to start with a small batch size like 8 combined with a quick one or two epochs which can depict memory consumption based on the batch size. YOLOv8.cls, the model used for YOLO classification, does not use too much memory so a batch size of 16 only uses 1.14 GB of memory. Equivalently, that means if you have 16 GB of RAM a batch size of 32 is acceptable.

In the case of YOLOv8n, the model used for YOLO detection rather than YOLO classification, memory usage with a batch size of 16 is 4.2GB and with a batch size of 32 is 7.5GB. Simplifying adjusting these two parameters improved our performance and enabled us to run experiments faster. For example, in the first run, using a stratified data split, we obtain these results:

classes	top1_acc	top5_acc: 100%	23/23	8.9it/s	2.6s<0.1s
All	0.881	0.997	In only 3m 47s		

in only a mere 3 minutes and 47 seconds for all runs. As part of the research and exploration, we tested different environments to conduct our experiments. We tested primarily on our local machines but we also experimented with using Google Colab.

Google Colab is free and offers a configurable environment to help to run model training with different options. There is GPU usage and it connects to a Google Drive so you do not need to hassle with preprocessing the dataset each time. The results are also saved in the Google Drive for future references. Depending on the model complexity, this was also an amazing option for expediting performance so we could gain results of our model faster.

What Did Not Work as Well:

Training unnecessary additional upper layers in the pretrained models, did not significantly improve our model's accuracy and was not worth the additional training time. We found that training around three of the upper layers gave us the best performance and accuracy. We found that this accuracy improved in where the model's pretrained dataset has not seen the classifier, such as the Miscellaneous Trash. Also, one of the biggest performance issues we faced was doing the prepping and all the training in one Jupyter Notebook. We managed to squeeze in three models but the Python Runtime took a great performance hit, where some cells took around 10 seconds to complete, while before it took less than a second. This performance hit made it hard to add additional features for our model comparisons. For this reason we chose to have the YOLO detection in a separate Notebook.

RealWaste is naturally a classification dataset, not a detection dataset. YOLO detection requires true bounding-box annotations which are done carefully by hand to ensure accuracy of the dataset. Therefore, OpenCV-generated boxes are approximate and can create noisy, inaccurate labels at times. Moreover, one box per image training data does not teach YOLO to detect multiple separate objects in different categories from a single image because no differentiation can occur between objects or classification types. As a workaround to this problem, we lowered confidence and increased the image size to help reveal detections, but it did not fully solve the multi-object issue. Additionally, Google Colab has a cap on GPU usage and if the hyper-parameters and performance parameters are not optimized, the training will require too much time and consequently will not be completed. This was a real barrier in training the YOLO detection model as at times changing any hyperparameter would require a substantial five hours of training only to retest that one hyperparameter that was changed. Using the options we discussed in the what worked section, we were able to mitigate this issue.

Image Classification Application

YOLO (You Only Look Once) [3] recent versions include utilities to simplify tasks such as training and exporting the model, making it very portable as a component in applications to predict or classify objects, in our project specifically waste types.

A trained model can be exported using the model's own utility. YOLOv8-cls and YOLOv8n, developed by Ultralytics [4], are available for academic purposes under AGPL-3.0 licenses. For our demo, we selected the Core ML framework from Apple. Apple portable devices are very popular and also include high-performance hardware integration (excellent camera, efficient memory, and multicore microprocessors). The trained prediction model size is 6.2 MB, and the classification model size is 2.9 MB. Our GoogLeNet model was easy to export to Core ML as well. The integration with the existing swift code was straightforward: just replace the input parameter YOLOv8 use image and GoogLeNet use input.

For our project the classic application is classification or prediction of trash types during various stages in waste management [8] like a robot picking trash from the beach or waste processing plants [9] sorting out waste before beginning the recycle process. An iPhone app can simulate what the robot or the sensors will see in a simplified simulation.

Conclusion

The strongest completed solution is the YOLOv8-cls classification because it reached a high accuracy quickly compared to the other models and it did not need any bounding boxes. Moreover, a test accuracy of 90% was reached in the data augmentation and stratified split version with only a total of 30 epochs. This training took merely minutes compared to the YOLO detection optimal model of a prediction of 89% which took a day to train and its training time was competitive to the other classification models while achieving a higher precision. The downside of the YOLOv8-cls model is that it has approximately a 10X in GFLOPs which may require more resources for an extra 3% accuracy. Another concern about the YOLO models is that it has a more restricted licence from Ultralytics compared to the other models which needs to be taken in high consideration when using the library for a project for non-educational purposes. Next, the PyTorch transfer-learning notebook served as a valuable baseline and helped us grasp the fundamentals of CNN concepts on a deeper level. Future work should include manually labeled bounding boxes or a real object-detection dataset before considering full YOLO multi-object detection.

Appendix:

Image Classification Application

YOLO (You Only Look Once) [3] recent versions include utilities to simplify tasks such as training and exporting the model, making it very portable as a component in applications to predict or classify objects, in our project specifically waste types.

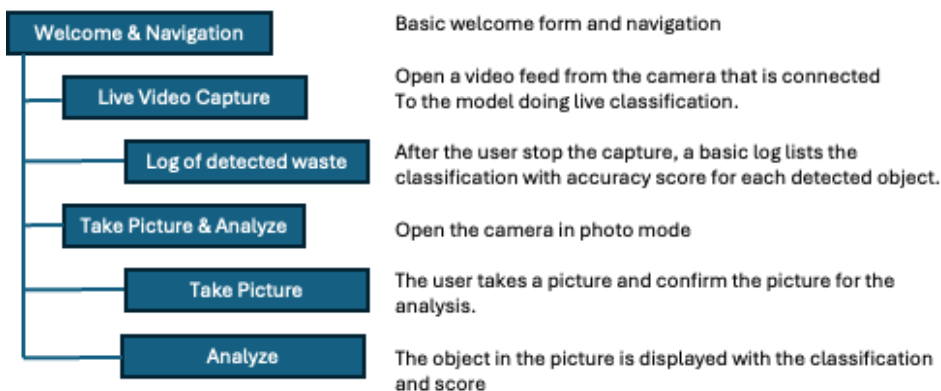
A trained model can be exported using the model's own utility. YOLOv8-cls and YOLOv8n, developed by Ultralytics [4], are available for academic purposes under AGPL-3.0 licenses. For our demo, we selected the Core ML framework from Apple. Apple portable devices are very popular and also include high-performance hardware integration (excellent camera, efficient memory, and multicore microprocessors). The trained prediction model size is 6.2 MB, and the classification model size is 2.9 MB.

For our project the classic application is classification or prediction of trash types during various stages during waste management [8] in communities. A robot picking trash from the beach, waste processing plants [9] sorting out waste before beginning the recycle process.

Python Code:

```
from ultralytics import YOLO
# Load your trained model
model = YOLO("/yolov8/runs/weights/best.pt")
# Export to Core ML
model.export(format="coreml")
```

To demonstrate our model using an iPhone app, we created this basic application design:



Xcode - Swift code using our models.

Take Picture & Analyze

YOLOv8

```
private func classifyImage(_ image: UIImage) {
    guard let pixelBuffer = image.toPixelBuffer(width: 224, height: 224) else {
        return
    }

    do {
```

```

let model = try YOLOv8cls(configuration: MLModelConfiguration())
let output = try model.prediction(image: pixelBuffer)

let label = output.classLabel
let confidence = output.classLabel_probs[label] ?? 0.0

resultLabel = label
resultConfidence = confidence

} catch {
    print("Prediction error:", error)
}
}
}

```

Live Capture - Camera Manager.

```

private let model: YOLOv8cls? = {
    do {
        return try YOLOv8cls(configuration: MLModelConfiguration())
    } catch {
        print("Model load error:", error)
        return nil
    }
}

```

GoogLeNet

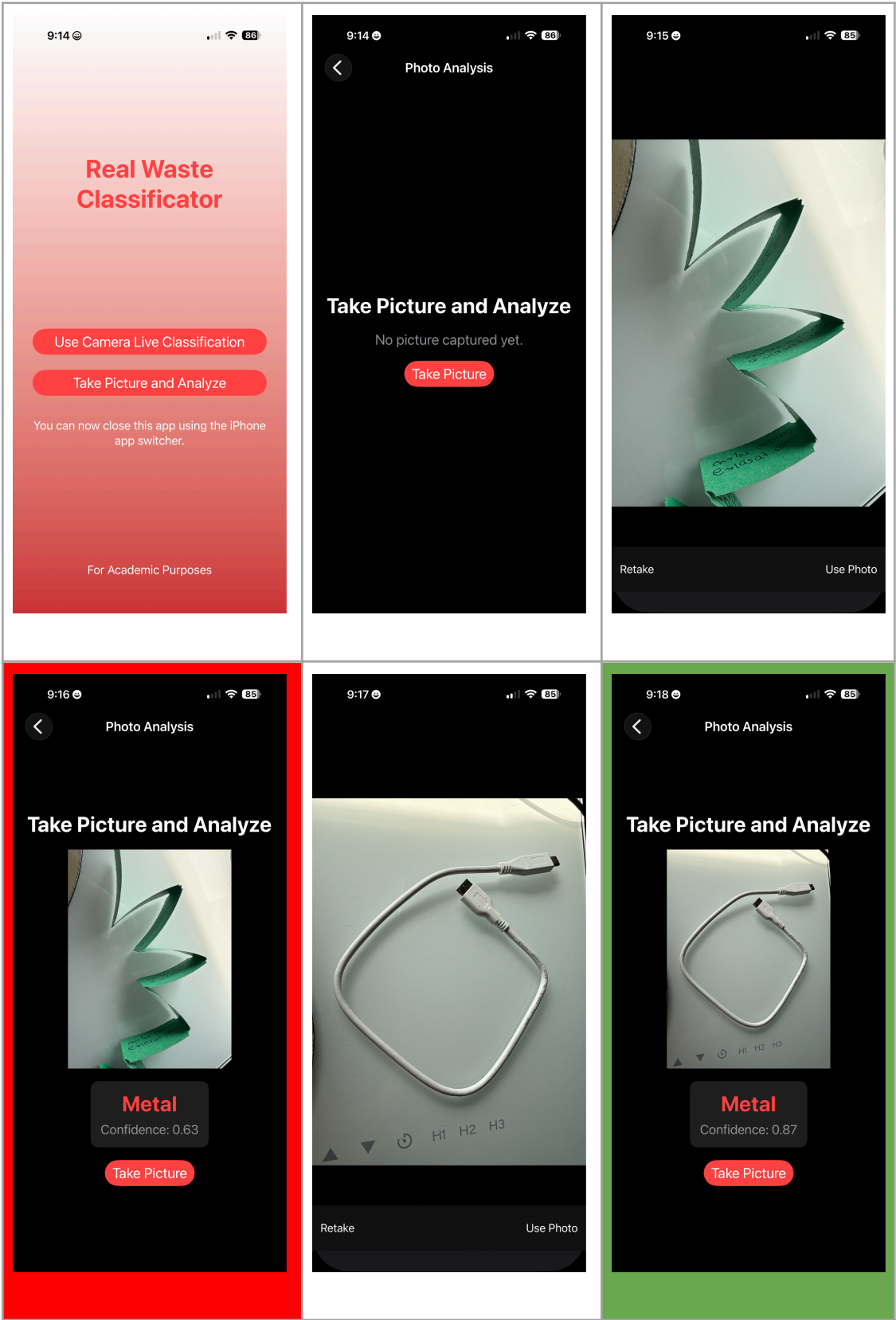
```

private let model: GoogLeNet? = {
    do {
        return try GoogLeNet(configuration: MLModelConfiguration())
    } catch {
        print("Model load error:", error)
        return nil
    }
}

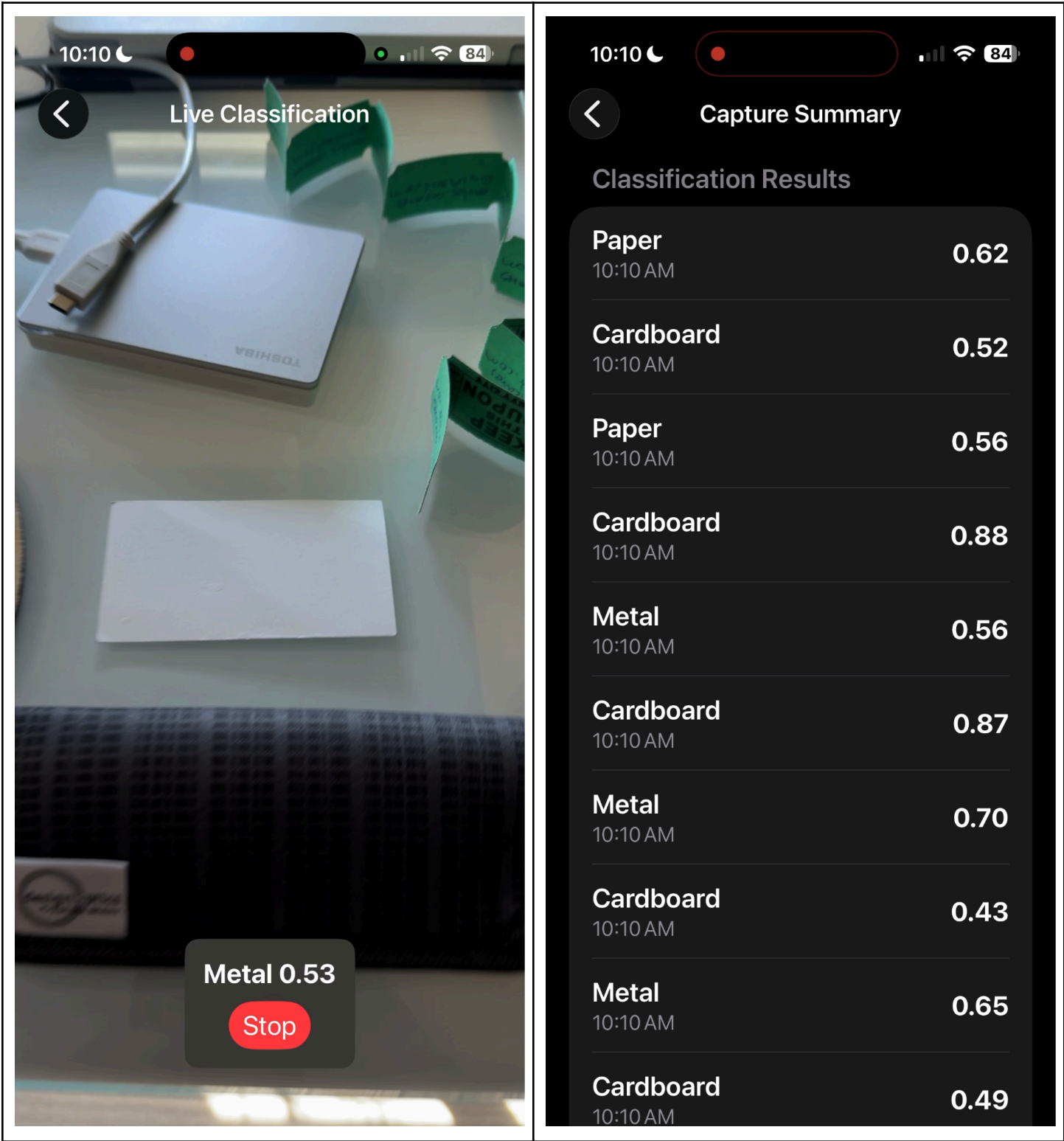
if self.session.canAddInput(input) {
    self.session.addInput(input)
} else {
    DispatchQueue.main.async {
        self.statusMessage = "Could not add camera input"
    }
    self.session.commitConfiguration()
    return
}

```


Screenshots from the app



Screenshot from the live video capture:



Project Links

Jupyter Notebook

Github Jupyter Notebooks: <https://github.com/juliareinhart/waste-classification-ml>

Machine Learning Libraries Used

PyTorch: <https://pytorch.org/>

TorchVision: <https://docs.pytorch.org/vision/stable/index.html>

Scikit-Learn: <https://scikit-learn.org/stable/>

Ultralytics: <https://docs.ultralytics.com/>

References

[1] Aurélien Géron, Hands-On Machine Learning with Scikit-Learn and Pytorch: O'Reilly Media, Inc, 2026.[Online]. Available: <https://learning.oreilly.com/library/view/hands-on-machine-learning/9798341607972/>. Accessed: May, 2026

[2] Sebastian Rashcka, Yuxi (Hyden) Liu, Vahid Mirjalili, Machine Learning with PyTorch and Scikit-Learn: Packt Publishing, 2022.[Online] Available: <https://learning.oreilly.com/library/view/machine-learning-with/9781801819312/>. Accessed May, 2026

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2016. [Online]. Available: <http://pjreddie.com/yolo/>

[4] Ultralytics. Ultralytics YOLO Docs. <https://docs.ultralytics.com/>. Accessed May, 2026

[5] Erik Linder-Norén. PyTorch-YOLOv3. <https://github.com/eriklindernoren/PyTorch-YOLOv3>. Accessed May 2026

[6] Sam Single, Saeid Iranmanesh, Raad Raad. RealWaste. <https://archive.ics.uci.edu/dataset/908/realwaste>. 2023. Accessed May 2026

[7] Suraj Subramanian, Seth Juarez, Cassie Breviu, Dmitry Soshnikov, Ari Bornstein. Learn the Basics. <https://docs.pytorch.org/tutorials/beginner/basics/intro.html>. 2026. Accessed May 2026

- [8] S. Kunkekar, "Harnessing machine learning for effective waste classification and recycling," *International Journal for Research in Applied Science and Engineering Technology*, vol. 12, no. 10, pp. 382–385, Oct. 2024, doi: 10.22214/ijraset.2024.64506.
- [9] J. Choi *et al.*, "Advancing plastic waste classification and recycling efficiency: Integrating image sensors and deep learning algorithms," *Applied Sciences*, vol. 13, no. 18, p. 10224, Sep. 2023, doi: 10.3390/app131810224.
- [10] PyTorch. Models and Pretrained Weights. <https://docs.pytorch.org/vision/main/models>. 2026. Accessed May 2026
- [11] PyTorch. torchvision.transforms. <https://docs.pytorch.org/vision/stable/transforms.html>. 2026. Accessed May 2026
- [12] PyTorch. RandomResizedCrop. <https://docs.pytorch.org/vision/main/generated/torchvision.transforms.RandomResizedCrop.html>. 2026. Accessed May 2026
- [13] PyTorch. RandomHorizontalFlip. <https://docs.pytorch.org/vision/main/generated/torchvision.transforms.RandomHorizontalFlip.html>. 2026. Accessed May 2026
- [14] PyTorch. ColorJitter. <https://docs.pytorch.org/vision/main/generated/torchvision.transforms.ColorJitter.html>. 2026. Accessed May 2026
- [15] PyTorch. RandomPerspective. <https://docs.pytorch.org/vision/main/generated/torchvision.transforms.RandomPerspective.html>. 2026. Accessed May 2026
- [16] CodeGenes.net. "How to Do Random Scale in PyTorch." <https://www.codegenes.net/blog/how-to-do-random-scale-pytorch/>. Accessed May 2026